Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Науково-навчальний інститут фізико-технічний інститут
Кафедра інформаційної безпеки

Лабораторна робота №3

з дисципліни «Технологія блокчейн та розподілені системи»

Виконали:
студенти групи ФБ-51мп
Спускан Дарина Михайлівна,
Свердлик Іван Васильович

Перевірила:
асистент кафедри ММЗІ
Селюх Поліна Валентинівна

Дата захисту _____

Захищено з балом _____

Київ 2026

ЗМІСТ

ВСТУП 3

РОЗДІЛ 1. ТЕОРЕТИЧНІ ОСНОВИ БЕЗПЕЧНОЇ РЕАЛІЗАЦІЇ ДЕЦЕНТРАЛІЗОВАНИХ ДОДАТКІВ НА БАЗІ ETHEREUM 5

1.1. Поняття децентралізованого додатку та його основні характеристики. 5

1.2. Архітектура децентралізованих додатків на платформі Ethereum. 9

1.3. Особливості функціонування смарт-додатків у мережі Ethereum. 14

1.4. Основні загрози та вразливості децентралізованих додатків. 18

РОЗДІЛ 2. АНАЛІЗ ВИМОГ OWASP ДО БЕЗПЕКИ WEB-ДОДАТКІВ 24

2.1. Призначення та структура стандарту OWASP TOP 10. 24

2.2. Характеристика основних категорій вразливостей OWASP. 28

2.3. Аналіз можливості застосування вимог OWASP до децентралізованих додатків. 30

2.4. Обмеження класичних підходів OWASP у контексті blockchain-систем. 33

РОЗДІЛ 3. ФОРМУВАННЯ АНАЛОГІЧНИХ ВИМОГ БЕЗПЕКИ ДЛЯ ДЕЦЕНТРАЛІЗОВАНИХ ДОДАТКІВ НА БАЗІ ETHEREUM 36

3.1. Адаптація вимог OWASP до архітектури Ethereum dApp. 36

3.2. Вимоги безпеки до клієнтської частини децентралізованого додатку. 38

3.3. Вимоги безпеки до взаємодії з криптогаманцями та підписом транзакцій. 41

3.4. Вимоги безпеки до смарт-контрактів Ethereum. 43

3.5. Вимоги безпеки до розгортання, адміністрування та супроводу dApp. 46

3.6. Узагальнений перелік вимог безпечної реалізації децентралізованих додатків на базі Ethereum. 49

ВИСНОВКИ 53

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 55

ВСТУП

У сучасному цифровому середовищі, що характеризується високими темпами розвитку інформаційних технологій та зростанням обсягів обробки даних, особливої актуальності набувають питання забезпечення інформаційної безпеки програмних систем. Традиційні централізовані підходи до побудови інформаційних систем поступово доповнюються або замінюються новими архітектурними моделями, серед яких особливе місце займають децентралізовані додатки, побудовані на базі blockchain-технологій. Такі додатки забезпечують відсутність єдиного центру керування, прозорість виконання операцій, незмінність збережених даних та підвищений рівень довіри між учасниками системи, що робить їх перспективними для використання у фінансовій сфері, електронному врядуванні, управлінні цифровими активами та багатьох інших галузях.

Однією з найбільш поширених платформ для створення децентралізованих додатків є Ethereum, яка забезпечує можливість виконання програмованої логіки у вигляді смарт-контрактів у межах розподіленої мережі. На відміну від класичних web-додатків, де основна бізнес-логіка зосереджена на серверній стороні, у децентралізованих додатках значна частина логіки переноситься до смарт-контрактів, які функціонують у середовищі Ethereum Virtual Machine. Така особливість зумовлює зміну підходів до проєктування, розробки та забезпечення безпеки програмних систем, оскільки помилки у смарт-контрактах можуть призводити до незворотних наслідків, включаючи втрату цифрових активів або порушення коректності роботи всієї системи.

Разом із перевагами децентралізовані додатки мають низку специфічних викликів у сфері безпеки. До них належать вразливості смарт-контрактів, ризики, пов’язані з управлінням криптографічними ключами, можливість маніпуляцій із транзакціями, атаки типу front-running, економічні атаки з використанням механізмів позик та ліквідності, а також проблеми, пов’язані з взаємодією між on-chain та off-chain компонентами системи. Крім того, важливу роль відіграє людський фактор, зокрема ризики фішингу, підміни інтерфейсів взаємодії з користувачем та неправильного підпису транзакцій. Таким чином, безпека децентралізованих додатків є комплексною задачею, яка охоплює як програмні, так і організаційні аспекти.

У сфері забезпечення безпеки традиційних web-додатків одним із найбільш авторитетних підходів є рекомендації, сформульовані організацією OWASP, зокрема перелік найбільш поширених ризиків, відомий як OWASP Top 10. Цей стандарт систематизує ключові вразливості, що виникають у web-додатках, та надає рекомендації щодо їх запобігання. Однак пряме застосування цих вимог до децентралізованих додатків є недостатнім, оскільки blockchain-середовище має принципово інші характеристики, такі як публічність даних, незмінність транзакцій, відсутність централізованого контролю та використання криптографічних механізмів автентифікації замість класичних моделей ідентифікації користувачів.

У зв’язку з цим виникає необхідність адаптації існуючих підходів до забезпечення безпеки, зокрема вимог OWASP, до специфіки децентралізованих додатків. Така адаптація передбачає аналіз того, які категорії вразливостей залишаються актуальними у середовищі Ethereum, які змінюють свою природу, а

також які нові класи загроз з'являються у зв'язку з використанням смарт-контрактів та децентралізованої інфраструктури. Важливим аспектом є також формування узагальненого переліку вимог безпеки, який враховує як класичні підходи до захисту web-додатків, так і специфічні особливості blockchain-технологій.

Актуальність даної роботи обумовлена необхідністю системного підходу до аналізу безпеки децентралізованих додатків та формування чітких рекомендацій щодо їх безпечної реалізації. З огляду на зростання кількості інцидентів, пов'язаних із вразливостями смарт-контрактів та експлуатацією недоліків у проєктуванні dApp (decentralized applications), розробка структурованих вимог безпеки є важливим завданням як для дослідників, так і для практикуючих розробників.

Метою лабораторної роботи є дослідження вимог OWASP до безпеки web-додатків та формування на їх основі аналогічних вимог безпеки для децентралізованих додатків, реалізованих на платформі Ethereum. Досягнення цієї мети передбачає вирішення низки взаємопов'язаних завдань, серед яких аналіз архітектури децентралізованих додатків, дослідження особливостей функціонування смарт-контрактів, вивчення основних категорій вразливостей OWASP, оцінка їх застосовності до blockchain-систем, а також розробка узагальненого підходу до забезпечення безпеки dApp.

РОЗДІЛ 1. ТЕОРЕТИЧНІ ОСНОВИ БЕЗПЕЧНОЇ РЕАЛІЗАЦІЇ ДЕЦЕНТРАЛІЗОВАНИХ ДОДАТКІВ НА БАЗІ ETHEREUM

1.1. Поняття децентралізованого додатку та його основні характеристики.

У сучасному розвитку інформаційних систем дедалі більшого значення набуває концепція децентралізації, яка передбачає відмову від єдиного центру зберігання, обробки та контролю даних на користь розподілених обчислювальних середовищ. Одним із найбільш практично значущих результатів розвитку цього підходу стало формування нового класу програмних систем - децентралізованих додатків. Децентралізований додаток, або dApp, є програмною системою, логіка функціонування якої частково або повністю реалізується в децентралізованому середовищі, зазвичай на базі blockchain-платформи. На відміну від традиційних клієнт-серверних застосунків, де керування всіма основними процесами здійснюється через центральний сервер або інфраструктуру, dApp використовує розподілену мережу вузлів, які спільно забезпечують виконання обчислень, зберігання стану системи та підтвердження змін.

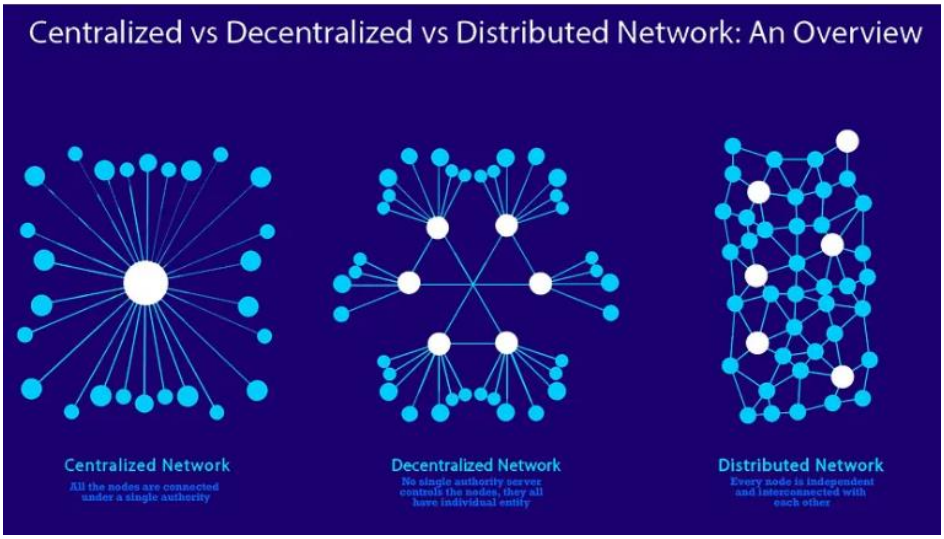

Рисунок 1. Порівняння топологій централізованої, децентралізованої та розподіленої мереж

Поява децентралізованих додатків стала логічним етапом розвитку blockchain-технологій. Якщо на початковому етапі блокчейн розглядався переважно як механізм зберігання та перевірки транзакцій у криптовалютних системах, то згодом його функціональність була значно розширена. Особливо важливу роль у цьому процесі відіграла платформа Ethereum, яка вперше надала широкі можливості для реалізації програмованої логіки у вигляді смарт-контрактів. Це дозволило створювати не лише платіжні системи, а й повноцінні прикладні сервіси, що виконують бізнес-логіку без участі централізованого сервера. Саме з цього моменту поняття децентралізованого додатку почало використовуватися не просто як теоретична модель, а як реальна архітектурна основа для побудови сучасних інформаційних систем.

Сутність децентралізованого додатку полягає в тому, що критично важлива логіка його функціонування, а також ключові дані, що визначають стан системи,

зберігаються та обробляються не в одному контрольованому середовищі, а у розподіленій мережі. У більшості випадків така логіка реалізується у формі смарт-контрактів - програмних модулів, які виконуються в blockchain-середовищі та автоматично реагують на виклики з боку користувачів або інших контрактів. У результаті виконання смарт-контракту змінюється стан блокчейну, а сама дія підтверджується мережею вузлів. Це означає, що результат роботи додатку не залежить від волі або коректності функціонування одного окремого адміністратора чи сервера, а визначається правилами, зафіксованими в коді та підтриманими консенсусом мережі.

Однією з ключових ознак децентралізованого додатку є відсутність єдиної точки довіри. У традиційних web-додатках користувач змушений довіряти власнику сервера, який забезпечує виконання логіки, зберігання даних, контроль доступу та адміністрування всієї системи. У dApp довіра переноситься з централізованого оператора на набір криптографічних механізмів, відкритий код смарт-контрактів та правила консенсусу мережі. Такий підхід дозволяє зменшити ризики зловживань з боку адміністратора, несанкціонованої модифікації даних або односторонньої зміни логіки роботи системи без відома користувачів. Водночас це означає, що питання правильності та безпеки коду набуває критичного значення, оскільки програмна логіка фактично замінює собою адміністративний контроль.

Ще однією фундаментальною характеристикою dApp є прозорість виконання операцій. У більшості публічних blockchain-мереж, зокрема в Ethereum, усі транзакції, виклики смарт-контрактів та зміни стану системи фіксуються в розподіленому реєстрі, доступному для перевірки будь-яким учасником мережі. Це створює умови для підвищення довіри до роботи системи, оскільки користувачі можуть незалежно перевіряти виконані операції, а також аналізувати поведінку контрактів та історію змін. У традиційних централізованих системах подібна прозорість зазвичай відсутня або досягається лише шляхом впровадження додаткових механізмів аудиту, які самі по собі потребують довіри до адміністратора. У dApp аудит є вбудованою властивістю самої архітектури.

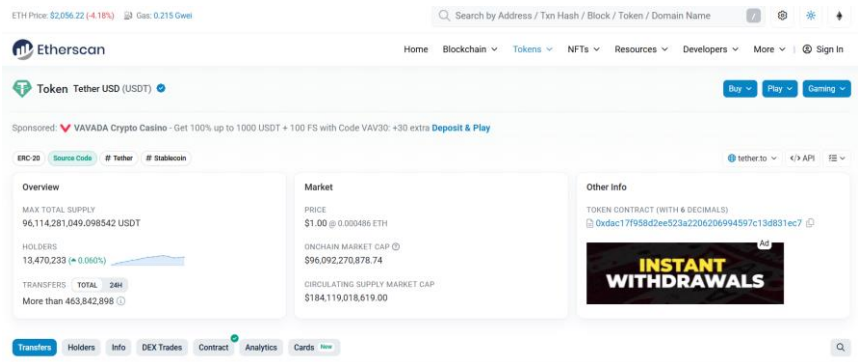

Рисунок 2. Офіційний оглядач блоків мережі (Etherscan), для прикладу взято відомий стейблкоїн USDT

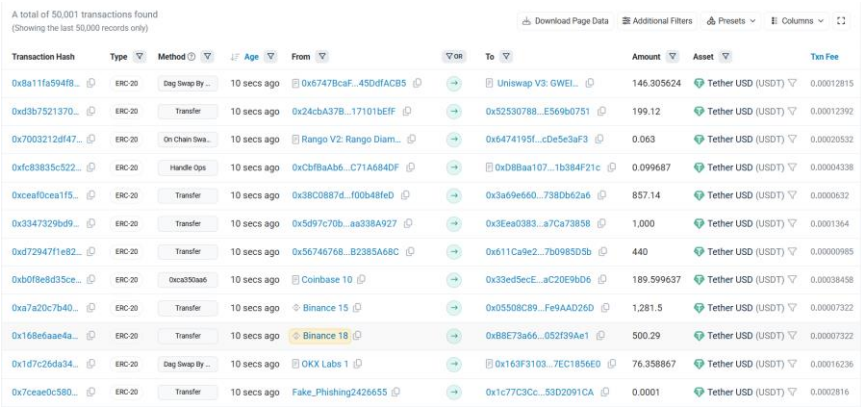

Рисунок 3. Розглянемо детальніше розділ “Transfers”

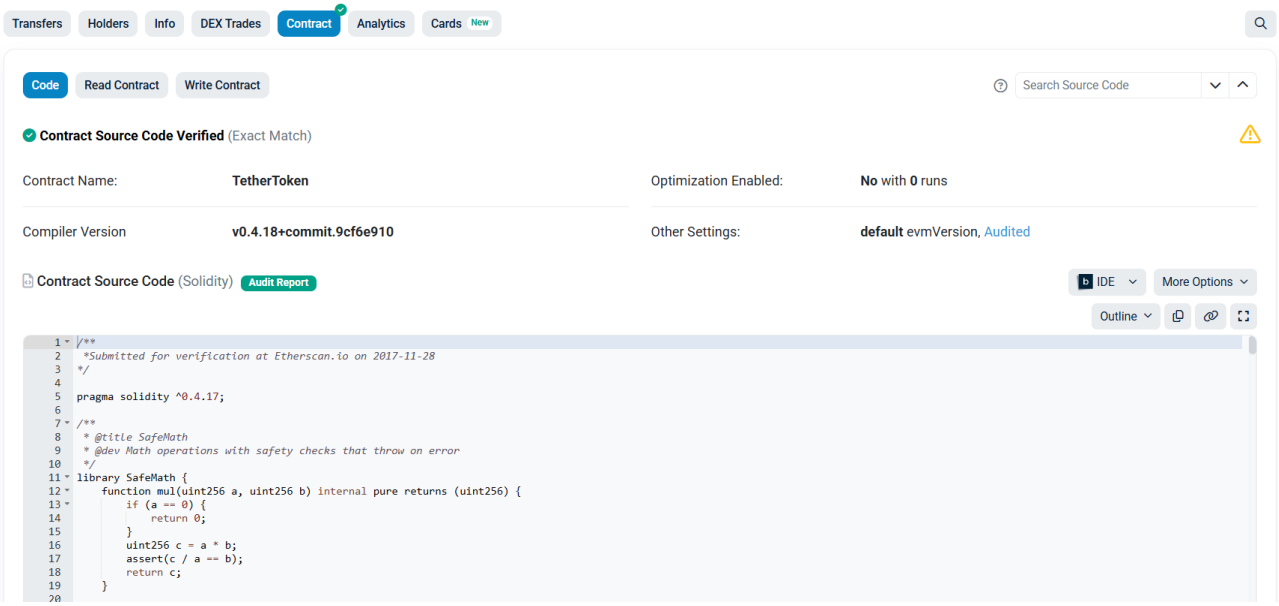

Рисунок 4. Розділ “Contract”, де знаходиться відкритий вихідний код смарт-контракту

Важливою особливістю децентралізованих додатків є також незмінність або обмежена змінність логіки після розгортання. У класичних web-системах розробник або адміністратор може у будь-який момент змінити код серверної частини, виправити помилку, змінити бізнес-правила або приховано модифікувати поведінку додатку. У випадку dApp, особливо тих, що працюють на базі смарт-контрактів у публічних blockchain-мережах, логіка після деплою стає або повністю незмінною, або змінюється лише через спеціально передбачені механізми оновлення. Це значно підвищує передбачуваність поведінки системи, але водночас створює серйозні вимоги до якості розробки, тестування та верифікації програмного коду до моменту його розгортання. Будь-яка помилка, закладена на етапі розробки, може залишитися в системі надовго або призвести до критичних наслідків.

Суттєвою характеристикою dApp є використання криптографічної ідентифікації замість класичних механізмів автентифікації. У звичайних web-додатках доступ до функціональності системи зазвичай здійснюється через логін, пароль, сесійні токени та інші централізовано контрольовані механізми. У децентралізованих додатках роль облікового запису виконує blockchain-адреса користувача, а підтвердження дій здійснюється шляхом цифрового підпису

транзакцій приватним ключем. Таким чином, користувач самостійно контролює свій доступ до активів і можливість взаємодії з додатком. Це значно зменшує залежність від централізованих механізмів автентифікації, але одночасно переносить відповідальність за збереження доступу безпосередньо на користувача. У разі компрометації приватного ключа або втрати seed-фрази наслідки можуть бути незворотними.

Характерною рисою децентралізованих додатків є те, що вони, як правило, функціонують у середовищі, де ресурси виконання мають вартісний вимір. У мережі Ethereum кожна транзакція та кожна операція смарт-контракту потребує витрат gas, тобто спеціального механізму оплати обчислювальних ресурсів мережі. Це відрізняє dApp від традиційних web-додатків, де більшість обчислень виконується на сервері без прямої участі користувача в оплаті кожної дії. У blockchain-середовищі користувач не лише взаємодіє з системою, а й фактично сплачує за її використання на рівні виконання операцій. Це накладає додаткові вимоги до оптимізації логіки смарт-контрактів, ефективності зберігання даних і продуманості архітектури системи.

Разом із тим важливо розуміти, що децентралізований додаток не завжди є повністю децентралізованим у всіх своїх компонентах. На практиці більшість сучасних dApp мають гібридну архітектуру, де частина логіки та даних зберігається on-chain, тобто безпосередньо в блокчейні, а інша частина реалізується off-chain - у вигляді веб-інтерфейсу, зовнішніх API, баз даних, серверів індексації, сервісів зберігання файлів або механізмів аналітики. Наприклад, інтерфейс користувача децентралізованого додатку часто залишається звичайним web-додатком, створеним за допомогою сучасних JavaScript-фреймворків, а взаємодія зі смарт-контрактами здійснюється через бібліотеки типу web3.js або ethers.js. Це означає, що dApp успадковує не лише нові blockchain-специфічні ризики, а й класичні проблеми web-безпеки, такі як міжсайтовий скриптинг, підміна інтерфейсу, компрометація клієнтського коду або небезпечна взаємодія із зовнішніми сервісами.

Залежно від функціонального призначення децентралізовані додатки можуть охоплювати дуже широкий спектр сфер застосування. До них належать фінансові сервіси децентралізованих фінансів, біржі цифрових активів, системи токенізації, маркетплейси цифрових активів, системи голосування, платформи для управління цифровою ідентичністю, сервіси децентралізованого зберігання інформації, системи управління ланцюгами постачання та багато інших прикладних рішень. Попри різноманітність сфер застосування, більшість таких систем мають спільні базові властивості: використання смарт-контрактів для реалізації критичної логіки, взаємодію з користувачами через криптогаманці, залежність від blockchain-інфраструктури та потребу в забезпеченні високого рівня довіри до виконуваних операцій.

У контексті інформаційної безпеки поняття децентралізованого додатку слід розглядати не лише як окремий тип програмного забезпечення, а як комплексну інформаційну систему з багаторівневою архітектурою та специфічною моделлю загроз. Якщо у традиційних web-додатках головний акцент часто робиться на захисті серверної логіки, бази даних і каналів взаємодії з користувачем, то в dApp безпека повинна охоплювати одночасно декілька площин: безпеку клієнтського інтерфейсу,

правильність та стійкість смарт-контрактів, захищеність механізмів підпису транзакцій, коректність роботи зовнішніх джерел даних, а також стійкість до атак, пов'язаних із публічністю та конкурентністю blockchain-середовища. У результаті децентралізований додаток є не просто альтернативою звичайному web-додатку, а окремою категорією програмних систем, яка потребує спеціалізованих підходів до аналізу, проєктування та забезпечення безпеки.

Отже, децентралізований додаток є програмною системою, що функціонує на базі розподіленої blockchain-інфраструктури та використовує смарт-контракти для реалізації критично важливої логіки. Його основними характеристиками є відсутність єдиного центру довіри, прозорість операцій, використання криптографічної моделі ідентифікації, часткова або повна незмінність програмної логіки після розгортання, а також поєднання on-chain та off-chain компонентів у межах єдиної архітектури. Саме ці особливості визначають як потенційні переваги децентралізованих додатків, так і специфіку загроз, які повинні враховуватися при їх безпечній реалізації.

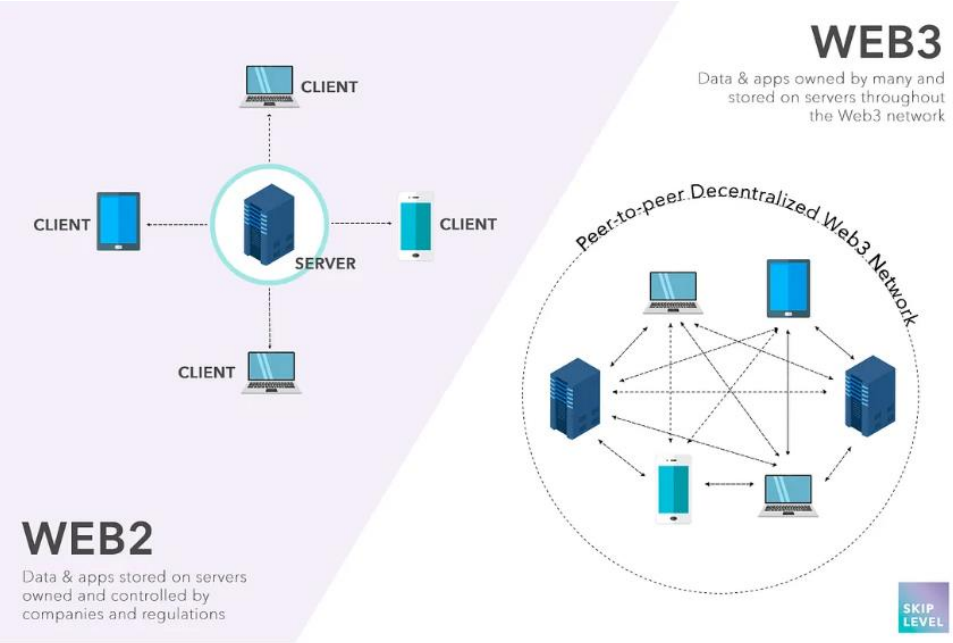

Рисунок 5. Архітектурні відмінності між традиційними WEB2 та децентралізованими WEB3

1.2. Архітектура децентралізованих додатків на платформі Ethereum.

Архітектура децентралізованих додатків на платформі Ethereum є багаторівневою та суттєво відрізняється від архітектури традиційних web-додатків. Якщо у класичній клієнт-серверній моделі користувач взаємодіє з веб-інтерфейсом, який надсилає запити на централізований сервер, а сервер уже обробляє логіку, звертається до бази даних та повертає результат, то в децентралізованому додатку значна частина критично важливої логіки виконується не на одному сервері, а в розподіленому середовищі blockchain-мережі. У випадку Ethereum таким середовищем виступає мережа вузлів, що підтримують виконання смарт-контрактів і

зберігають єдиний узгоджений стан системи. Саме тому архітектуру dApp доцільно розглядати як поєднання клієнтського рівня, blockchain-рівня, рівня взаємодії з користувачем через криптографічні інструменти, а також допоміжних off-chain компонентів, які забезпечують повноцінне функціонування системи.

Загалом архітектура децентралізованого додатку на платформі Ethereum складається з кількох логічно пов'язаних частин. До них належать клієнтський інтерфейс користувача, криптогаманець або інший інструмент підпису транзакцій, смарт-контракти, мережа Ethereum, вузли доступу до блокчейну та, за необхідності, зовнішні сервіси для зберігання даних або отримання інформації з позаблокчейнового середовища. Незважаючи на те, що в популярному уявленні dApp часто асоціюється лише зі смарт-контрактом, на практиці він є комплексною інформаційною системою, де кожен із зазначених компонентів виконує окрему функцію та має власні особливості з точки зору безпеки, надійності та функціональності.

Першим рівнем архітектури dApp є користувацький інтерфейс, або frontend. Саме через нього користувач взаємодіє із системою, переглядає доступні функції, ініціює операції, отримує результати та підписує необхідні транзакції. У більшості випадків frontend децентралізованого додатку реалізується як звичайний web-додаток із використанням сучасних технологій клієнтської розробки, зокрема JavaScript-фреймворків, таких як React, Vue або Angular. Зовні такий інтерфейс може майже не відрізнятися від звичайного вебсайту або SaaS-сервісу, однак його принципова особливість полягає в тому, що він не звертається до централізованого backend-сервера для виконання критично важливих операцій, а взаємодіє безпосередньо або опосередковано з blockchain-інфраструктурою. Користувацький інтерфейс dApp виконує кілька важливих функцій. По-перше, він відображає користувачеві стан системи, який може бути отриманий шляхом читання даних зі смарт-контрактів. По-друге, він надає механізми ініціювання дій, які потребують запису в блокчейн, наприклад створення заявки, переказу токенів, взаємодії з протоколом або голосування. По-третє, він організовує комунікацію між користувачем і криптогаманцем, через який відбувається підпис транзакцій. Отже, frontend у dApp не лише забезпечує зручність використання системи, а й виступає важливим елементом безпечної взаємодії між людиною та blockchain-логікою. Саме на цьому рівні можуть виникати загрози підміни адрес, фішингових інтерфейсів, маніпулятивного відображення даних або нав'язування користувачу небезпечних дій.

Наступним важливим компонентом архітектури є криптогаманець користувача. У традиційних web-додатках автентифікація та авторизація зазвичай реалізуються через систему облікових записів, паролів, токенів сесій або федеративних механізмів входу. У децентралізованих додатках на Ethereum роль користувацької ідентичності виконує blockchain-адреса, а підтвердження права на виконання дії здійснюється за допомогою цифрового підпису приватним ключем. Саме тому криптогаманець є не просто засобом зберігання цифрових активів, а повноцінним елементом архітектури додатку. Гаманець забезпечує кілька ключових функцій. Він зберігає або надає доступ до приватного ключа користувача, дозволяє підписувати транзакції, взаємодіє з веб-інтерфейсом через спеціальні API або

розширення браузера, а також інформує користувача про зміст операції, яку той має підтвердити. Фактично криптогаманець виконує роль довіреного криптографічного посередника між користувачем і блокчейном. Саме через нього відбувається підтвердження того, що певна дія була ініційована власником конкретної адреси. Водночас цей компонент є критично важливим із точки зору безпеки, оскільки компрометація гаманця або обман користувача під час підпису транзакції можуть призвести до безповоротної втрати контролю над активами або правами доступу.

Центральним елементом архітектури dApp на платформі Ethereum є смарт-контракти. Вони являють собою програмний код, що розгортається в мережі Ethereum та виконується в середовищі Ethereum Virtual Machine. Саме смарт-контракти визначають основну бізнес-логіку децентралізованого додатку, правила зміни стану системи, механізми доступу до функцій, обробку активів, збереження структур даних і взаємодію між різними частинами системи. У традиційній архітектурі аналогічну роль виконує серверна частина або backend, однак на відміну від неї смарт-контракт не контролюється одним оператором, а виконується всією мережею відповідно до узгоджених правил. Смарт-контракти можуть бути простими або багатокомпонентними. У простому випадку один контракт реалізує всі основні функції системи. У більш складних архітектурах dApp логіка розподіляється між кількома контрактами: один може відповідати за зберігання стану, інший - за обчислювальну логіку, ще інший - за контроль доступу або інтеграцію з токенами. У сучасних системах також поширені шаблони модульної архітектури, проксі-контракти, бібліотеки та окремі контракти-реєстри. Такий підхід підвищує гнучкість і масштабованість системи, але водночас ускладнює її аналіз і створює додаткові ризики помилок у взаємодії між компонентами. Важливою особливістю смарт-контрактів є те, що вони не виконуються самостійно “у фоновому режимі” в тому ж сенсі, як це може відбуватися з серверними процесами у традиційних системах. Їх виконання ініціюється транзакціями або викликами з боку інших контрактів. Це означає, що будь-яка зміна стану системи в dApp повинна бути чітко визначена як послідовність викликів функцій смарт-контрактів, підтверджена мережею та записана до блокчейну. Такий механізм забезпечує відтворюваність і перевірюваність усіх змін, але одночасно створює обмеження щодо швидкості виконання, вартості операцій та гнучкості бізнес-логіки.

Наступним рівнем архітектури є безпосередньо мережа Ethereum як середовище виконання та збереження стану. Ethereum являє собою розподілену мережу вузлів, кожен з яких підтримує копію актуального стану блокчейну та перевіряє коректність виконуваних транзакцій. Саме ця мережа забезпечує консенсус щодо того, які зміни вважаються допустимими, які транзакції є дійсними та яким є поточний стан смарт-контрактів. Із точки зору dApp Ethereum виконує роль спільного обчислювального та реєстраційного середовища, яке не належить окремому розробнику чи користувачу, але підтримує логіку додатку як частину загальної інфраструктури. Особливістю такого підходу є те, що мережа Ethereum виступає одночасно і середовищем виконання коду, і системою зберігання критично важливих даних. Це суттєво відрізняє її від традиційних баз даних або серверів додатків. Кожна зміна стану системи повинна бути підтверджена мережею, а кожне обчислення має свою вартість у вигляді витрат gas. Унаслідок цього архітектура

dApp повинна бути спроєктована таким чином, щоб у блокчейн виносилися лише ті частини логіки та даних, які дійсно потребують високого рівня довіри, незмінності та прозорості. Усе інше зазвичай реалізується на off-chain рівні, що дозволяє зменшити витрати та підвищити продуктивність.

Для того щоб frontend або інші зовнішні компоненти могли взаємодіяти з блокчейном, використовуються вузли доступу до мережі Ethereum або спеціалізовані RPC-сервіси. Саме через них клієнтський інтерфейс отримує інформацію про поточний стан блокчейну, баланси адрес, історію транзакцій, стан контрактів, а також надсилає запити на виконання читання чи ініціює відправлення транзакцій. У деяких випадках розробники або організації можуть підтримувати власний повноцінний вузол Ethereum, однак на практиці дуже часто використовуються зовнішні провайдери доступу до мережі. Вони надають інтерфейси взаємодії з блокчейном у зручній для застосунку формі та значно спрощують розробку і масштабування системи.

Незважаючи на те, що використання зовнішніх RPC-провайдерів спрощує архітектуру, воно одночасно створює додаткові залежності та ризики. Якщо такий сервіс недоступний, працює некоректно або повертає спотворені дані, це може вплинути на функціонування dApp, особливо на рівні відображення стану системи користувачу. Таким чином, навіть у децентралізованих системах можуть зберігатися елементи фактичної централізації, які необхідно враховувати при аналізі безпеки та надійності архітектури.

Окреме місце в архітектурі Ethereum dApp займають off-chain компоненти. Незважаючи на те, що основна довірена логіка реалізується у смарт-контрактах, повністю перенести весь функціонал у блокчейн зазвичай неможливо або економічно недоцільно. Причиною цього є як технічні обмеження blockchain-середовища, так і висока вартість зберігання великих обсягів даних та виконання складних обчислень. Саме тому більшість сучасних dApp використовують додаткові зовнішні сервіси, які працюють поза блокчейном, але інтегруються з ним.

До таких компонентів можуть належати системи зберігання файлів, індексаційні сервіси, API-шлюзи, бази даних, аналітичні сервіси, системи сповіщень, сервери для генерації метаданих, а також сервіси, що забезпечують доступ до зовнішньої інформації. Наприклад, якщо dApp працює з цифровими активами, такими як NFT, сам токен та право власності на нього можуть бути зафіксовані у блокчейні, але зображення, опис або інші пов’язані файли можуть зберігатися поза блокчейном. Аналогічно, інтерфейс може використовувати окремі індексаційні сервіси для швидкого пошуку та агрегації інформації, яку було б незручно отримувати безпосередньо з блокчейну в режимі реального часу.

Важливою частиною архітектури багатьох Ethereum dApp є також механізми інтеграції з зовнішнім світом через так звані oracle-рішення. Оскільки смарт-контракти самі по собі не можуть безпосередньо звертатися до зовнішніх джерел інформації, для отримання даних про курси активів, погодні умови, результати подій або інші параметри використовуються спеціальні сервіси-посередники. Вони доставляють перевірену або погоджену інформацію до блокчейну, де вона вже може використовуватися смарт-контрактами для прийняття рішень. Це значно розширює можливості dApp, але одночасно створює новий клас ризиків, пов’язаних із довірою

до джерела даних, можливістю маніпуляції зовнішньою інформацією та залежністю логіки контрактів від коректності роботи таких сервісів. Архітектура децентралізованого додатку на Ethereum також має важливу функціональну відмінність від традиційних web-систем у частині розподілу відповідальності між користувачем і системою. У класичних централізованих рішеннях значна частина відповідальності за захист облікового запису, обробку помилок, контроль доступу та відновлення стану лежить на серверній стороні. У dApp користувач самостійно відповідає за володіння ключами, підтвердження транзакцій і розуміння того, яку саме дію він виконує. Це означає, що архітектура dApp повинна бути не лише технічно коректною, а й достатньо прозорою та зрозумілою для кінцевого користувача, оскільки багато рішень щодо безпеки фактично делегуються йому.

У практичному аспекті архітектура Ethereum dApp може реалізовуватися в різних варіаціях залежно від призначення системи, однак її базова модель залишається відносно сталою. Користувач взаємодіє з web-інтерфейсом, який через бібліотеки та RPC-виклики отримує інформацію з блокчейну. Якщо користувач бажає виконати дію, що змінює стан системи, frontend формує відповідний запит на транзакцію, передає його до криптогаманця, після чого користувач підписує цю транзакцію своїм приватним ключем. Далі транзакція надсилається до мережі Ethereum, проходить валідацію, включається до блоку та змінює стан смарт-контракту. Після цього оновлений стан знову відображається у frontend. Таким чином, життєвий цикл взаємодії в dApp є більш складним, ніж у традиційному web-додатку, але водночас більш прозорим і криптографічно підтвердженим.

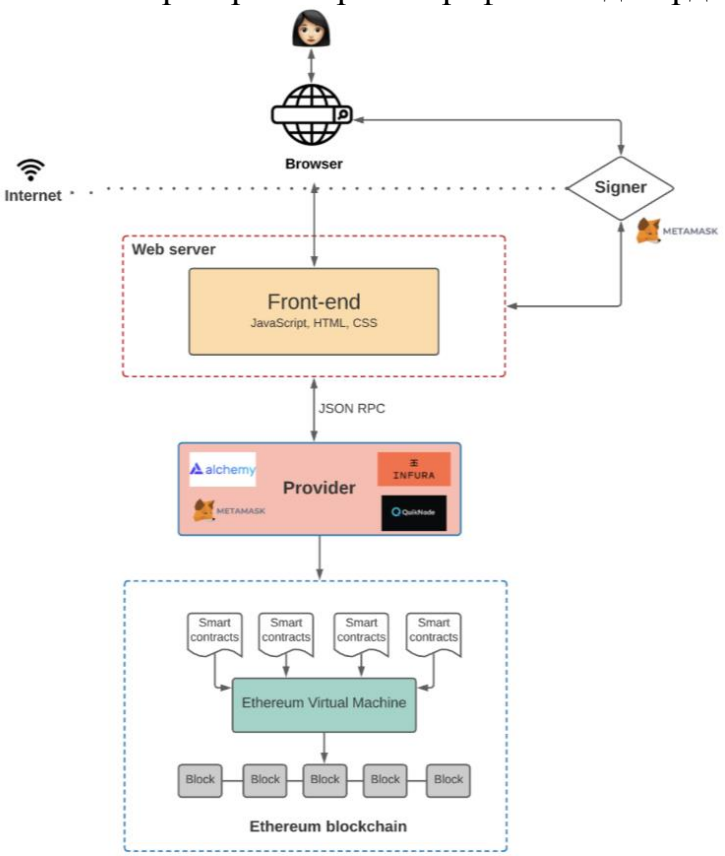

Рисунок 6. Схема взаємодії компонентів в архітектурі децентралізованого додатку на базі Ethereum

З точки зору безпеки саме архітектурна багатошаровість dApp визначає

складність їх захисту. Уразливість або помилка на будь-якому рівні - у frontend, у взаємодії з гаманцем, у смарт-контракті, у конфігурації RPC-доступу, у зовнішньому oracle-сервісі або в off-chain компоненті - може вплинути на загальну безпеку системи. При цьому навіть якщо смарт-контракт реалізовано бездоганно, dApp все одно може залишатися вразливим через слабку реалізацію користувацького інтерфейсу або небезпечну логіку взаємодії з підписом транзакцій. Саме тому архітектуру децентралізованих додатків на Ethereum необхідно розглядати як цілісну багатокомпонентну систему, безпечність якої визначається не лише на рівні blockchain-коду, а на рівні всієї екосистеми її функціонування.

Отже, архітектура децентралізованого додатку на платформі Ethereum є поєднанням кількох взаємопов'язаних компонентів: користувацького web-інтерфейсу, криптогаманця, смарт-контрактів, мережі Ethereum, вузлів доступу до блокчейну та допоміжних off-chain сервісів. Саме взаємодія цих компонентів забезпечує функціонування dApp як повноцінної інформаційної системи. Водночас така багаторівнева побудова створює не лише нові функціональні можливості, а й нові ризики, які необхідно враховувати при аналізі та формуванні вимог до безпечної реалізації децентралізованих додатків.

1.3. Особливості функціонування смарт-додатків у мережі Ethereum.

Смарт-контракти є ключовим елементом функціонування децентралізованих додатків на платформі Ethereum, оскільки саме вони реалізують основну бізнес-логіку системи, визначають правила взаємодії між учасниками та забезпечують автоматизоване виконання операцій у розподіленому середовищі. На відміну від звичайних програмних модулів серверної частини традиційного web-додатку, смарт-контракт функціонує не в межах одного контрольованого сервера, а в мережі Ethereum, де його код і поточний стан підтримуються множиною вузлів. Це означає, що логіка, закладена у смарт-контракті, виконується відповідно до єдиних правил мережі, а результат такого виконання визнається дійсним лише після досягнення консенсусу між учасниками blockchain-середовища.

У загальному розумінні смарт-контрактом називають програму, яка зберігається в блокчейні та виконує заздалегідь визначені дії за настання певних умов. У контексті Ethereum смарт-контракт є набором функцій і структур даних, розгорнутих за окремою blockchain-адресою та доступних для виклику іншими користувачами, додатками або контрактами. Після розгортання контракт отримує власну адресу в мережі, а його код стає частиною blockchain-інфраструктури. У подальшому взаємодія з контрактом здійснюється через виклики його функцій, які можуть або лише зчитувати дані, або змінювати стан системи шляхом створення транзакцій.

Функціонування смарт-контрактів у мережі Ethereum безпосередньо пов'язане з роботою Ethereum Virtual Machine. EVM є спеціалізованим середовищем виконання, яке забезпечує однакову інтерпретацію програмного коду на всіх вузлах мережі. Це означає, що будь-який смарт-контракт виконується детерміновано: за однакових вхідних даних і однакового початкового стану всі вузли повинні отримати однаковий результат. Саме ця властивість є критично важливою для підтримання цілісності мережі, оскільки будь-яка невизначеність або неоднозначність виконання призвела б до розбіжностей у стані блокчейну та

порушення консенсусу. Тому код смарт-контракту в Ethereum має бути не просто функціонально правильним, а й сумісним із суворими вимогами до відтворюваності виконання.

Розробка смарт-контрактів зазвичай здійснюється мовами високого рівня, найпоширенішою серед яких є Solidity. Після написання вихідного коду він компілюється у байткод, який уже розгортається в мережі Ethereum і виконується середовищем EVM. Такий підхід нагадує компіляцію звичайних програм, однак має принципову відмінність: після розгортання байткод стає частиною публічної та розподіленої інфраструктури, а кожна його операція потенційно виконується великою кількістю вузлів. У результаті навіть відносно невелика логічна помилка, яка в централізованому застосунку могла б бути виправлена оновленням коду на сервері, у blockchain-середовищі може зберігатися в системі тривалий час і мати набагато серйозніші наслідки.

Смарт-контракт у мережі Ethereum функціонує як об’єкт зі станом. Це означає, що він не лише виконує певні обчислення, а й зберігає дані між викликами. До таких даних можуть належати баланси, списки користувачів, параметри конфігурації, результати голосувань, адреси адміністраторів, записи про права доступу та будь-яка інша інформація, необхідна для роботи конкретного децентралізованого додатку. Стан контракту зберігається безпосередньо в блокчейні, а зміна цього стану можлива лише через виконання функцій, які були передбачені розробником у коді. Таким чином, смарт-контракт виступає одночасно і як програмний модуль, і як сховище критично важливої інформації.

Усі взаємодії зі смарт-контрактом можна умовно поділити на дві великі категорії. Перша категорія - це виклики, які не змінюють стан блокчейну. Вони використовуються для читання вже наявних даних і не потребують створення транзакції. Наприклад, користувач може запитати поточний баланс токена, перевірити значення певної змінної або отримати відомості про стан голосування. Такі операції зазвичай виконуються локально через RPC-виклик до вузла Ethereum і не вимагають оплати gas у тому сенсі, як це відбувається для транзакцій, що змінюють стан. Друга категорія - це виклики, які змінюють стан контракту або блокчейну в цілому. Саме вони оформлюються як транзакції, підписуються приватним ключем користувача, надсилаються до мережі, проходять перевірку та включаються до блоку.

Транзакція в Ethereum є базовим механізмом ініціювання змін у системі. Якщо користувач бажає виконати дію, яка впливає на стан контракту, наприклад переказати токени, зареєструвати запис, змінити параметр або проголосувати, він повинен сформувати транзакцію, що містить адресу контракту, виклик потрібної функції та необхідні параметри. Ця транзакція підписується приватним ключем користувача, що підтверджує її автентичність і право на виконання дії від імені конкретної адреси. Після цього транзакція поширюється мережею, перевіряється вузлами та в разі коректності включається до нового блоку. Лише після цього зміни вважаються підтвердженими та відображаються у глобальному стані мережі.

Окремою особливістю функціонування смарт-контрактів є використання механізму gas. У середовищі Ethereum кожна операція, що виконується в EVM, має певну обчислювальну вартість. Gas виконує роль універсальної одиниці

вимірювання цієї вартості та одночасно механізму захисту мережі від надмірного або нескінченного виконання коду. Коли користувач надсилає транзакцію, він повинен визначити, яку кількість gas готовий витратити на її виконання. Якщо під час виконання контракту відведеного обсягу gas виявляється недостатньо, транзакція завершується з помилкою, а зміни стану не застосовуються, хоча самі витрати на виконання при цьому можуть бути втрачені. Такий механізм стимулює розробників до написання оптимізованого коду, а також обмежує можливість зловживань обчислювальними ресурсами мережі.

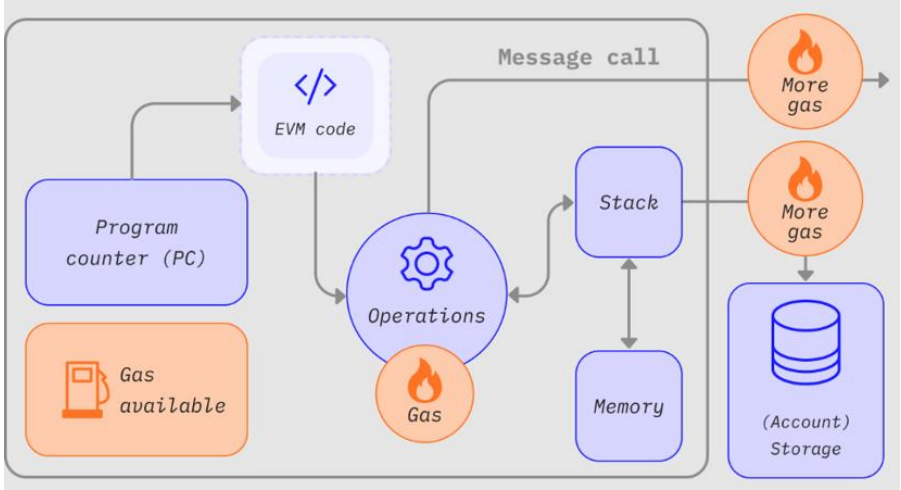

Рисунок 7. Процес виконання коду смарт-контракту EVM та споживання gas

Газова модель має не лише технічне, а й безпекове значення. Вона змушує враховувати ресурсоємність кожної функції смарт-контракту, особливо якщо йдеться про цикли, обробку великих масивів даних або складну багатокрокову логіку. Якщо функція спроєктована неефективно, її виконання може стати занадто дорогим або навіть неможливим у межах практично допустимого обсягу gas. Це створює ризики відмови в обслуговуванні на рівні бізнес-логіки, коли контракт формально є правильним, але деякі його функції фактично не можуть бути використані через надмірну вартість виконання. Отже, особливість функціонування смарт-контрактів у мережі Ethereum полягає в тому, що безпека, коректність і ефективність коду є взаємопов'язаними характеристиками.

Суттєвою рисою смарт-контрактів є їхня публічність. Після розгортання код контракту, його адреса, виклики функцій, зміни стану та історія транзакцій можуть бути проаналізовані учасниками мережі. Навіть якщо вихідний код не був опублікований окремо, байткод і результати виконання все одно доступні для дослідження. Це означає, що смарт-контракт слід розглядати як таку програму, логіка якої потенційно видима для зловмисника. У традиційних централізованих системах внутрішня логіка серверної частини зазвичай прихована від зовнішнього користувача, що певною мірою ускладнює аналіз атакувальної поверхні. У випадку Ethereum подібної "прихованості" фактично немає, тому будь-яка хибна логіка, недостатній контроль доступу або неврахована бізнес-умова можуть бути виявлені та використані сторонніми особами.

Ще однією особливістю є незворотність або висока складність виправлення помилок після розгортання контракту. Якщо в традиційному web-додатку розробник

виявляє помилку, він може змінити код на сервері, повторно розгорнути систему або встановити оновлення. У випадку зі смарт-контрактами ситуація є значно складнішою. Якщо контракт уже розгорнуто в мережі та активно використовується, безпосередньо змінити його код зазвичай неможливо. Для цього необхідно або передбачати спеціальні механізми оновлення заздалегідь, або розгортати новий контракт і переносити на нього логіку та користувачів. Унаслідок цього до етапу тестування, формальної перевірки та аудиту контрактів висуваються набагато жорсткіші вимоги, ніж до звичайних програмних модулів.

Функціонування смарт-контрактів у мережі Ethereum також характеризується тим, що вони можуть взаємодіяти один з одним. Один контракт здатний викликати функції іншого контракту, передавати йому значення, отримувати відповідь, працювати з токенами, делегувати частину логіки або виступати посередником у складніших бізнес-процесах. Така можливість значно розширює архітектурні можливості децентралізованих систем, дозволяючи будувати модульні та композиційні рішення. Водночас саме міжконтрактна взаємодія часто стає джерелом складних вразливостей. Помилки в порядку викликів, неправильна обробка зовнішніх викликів, відсутність перевірки результатів або довіра до неперевірених контрактів можуть призвести до критичних наслідків. Таким чином, контракт у Ethereum рідко функціонує ізольовано, а отже його безпечність значною мірою залежить і від оточення, у якому він працює.

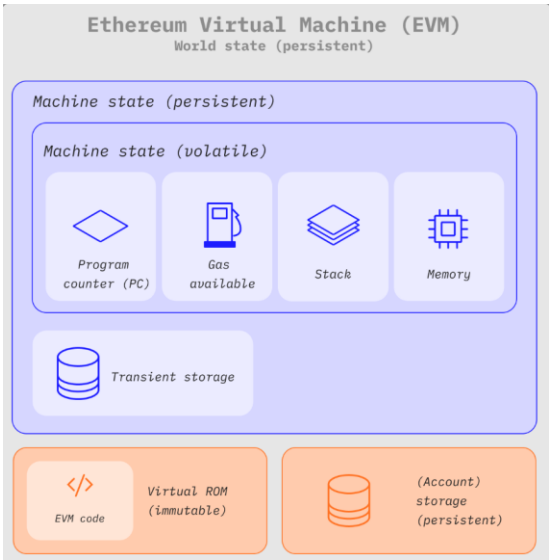

Рисунок 8. Внутрішня архітектура та структура стану EVM

Особливу увагу слід приділити механізму виклику функцій та зміні стану контракту. Кожна функція має власний рівень доступності та може бути або публічною, або внутрішньою, або доступною лише в певних умовах. У правильно спроєктованому контракті критичні функції повинні мати чітко визначені умови виклику, наприклад перевірку того, що дія виконується лише власником контракту, адміністратором або учасником із відповідними правами. Якщо така перевірка відсутня або реалізована помилково, будь-який зовнішній користувач може отримати змогу змінювати стан системи несанкціонованим чином. Саме тому особливість функціонування смарт-контрактів полягає не лише в автоматизованому

виконанні коду, а й у тому, що будь-яка помилка в логіці доступу негайно перетворюється на практичну вразливість.

Важливо також враховувати, що смарт-контракти працюють у конкурентному та відкритому транзакційному середовищі. Після надсилання транзакції користувач не має абсолютної гарантії того, в якому саме порядку вона буде включена до блоку відносно інших транзакцій. Це створює передумови для так званих фронтранінгових сценаріїв, коли інші учасники мережі можуть побачити транзакцію до її підтвердження й спробувати виконати власну операцію раніше або з вигіднішими параметрами. У традиційних централізованих системах подібна проблема зазвичай вирішується на рівні сервера та контролю черги операцій. У середовищі Ethereum питання порядку виконання транзакцій має значно складніший характер і повинно враховуватися ще на етапі проєктування смарт-контракту.

Ще властивістю смарт-контрактів є їхня обмежена здатність працювати із зовнішнім середовищем. На відміну від серверної програми, контракт не може безпосередньо виконувати HTTP-запити, читати файли, отримувати випадкові значення з операційної системи або звертатися до сторонніх API. Усі його дії повинні бути детермінованими та придатними до відтворення на кожному вузлі мережі. Саме тому смарт-контракти є ізольованими від зовнішнього світу і можуть отримувати зовнішню інформацію лише через спеціальні механізми, наприклад через oracle-сервіси. Це обмеження є принциповим для розуміння логіки їх функціонування, оскільки воно визначає архітектурні межі того, що можна безпечно реалізувати безпосередньо в блокчейні.

Крім того, функціонування смарт-контрактів слід розглядати в контексті економічної моделі Ethereum. Будь-яка дія в системі має не лише технічний, а й фінансовий вимір. Якщо в традиційному вебдодатку помилкова або зайва операція може призвести лише до надлишкового навантаження на сервер, то в Ethereum неефективно написана функція або нераціональна архітектура можуть безпосередньо збільшувати витрати користувачів. Внаслідок цього оцінка коректності смарт-контракту повинна включати не лише його логічну правильність, але й аналіз економічної доцільності його функціонування. Саме тому в середовищі Ethereum безпека часто тісно переплітається з економікою протоколу, а деякі атаки мають не суто технічний, а фінансово-маніпулятивний характер.

1.4. Основні загрози та вразливості децентралізованих додатків.

Попри значні переваги децентралізованих додатків, пов’язані з прозорістю, незмінністю логіки та відсутністю єдиного центру контролю, їх функціонування супроводжується широким спектром специфічних загроз та вразливостей. На відміну від традиційних web-додатків, де основний акцент безпеки зосереджується на захисті серверної логіки, баз даних, сесій та механізмів автентифікації, у децентралізованих додатках атакувальна поверхня є значно ширшою. Вона охоплює не лише клієнтський інтерфейс і логіку обробки даних, а й смарт-контракти, механізми підпису транзакцій, взаємодію з blockchain-мережею, зовнішні джерела даних, інфраструктурні сервіси та економічну модель самої системи. Саме тому загрози безпеці dApp мають комплексний характер і часто виходять за межі класичних уявлень про вразливості програмного забезпечення.

Однією з найнебезпечніших категорій загроз у децентралізованих додатках є

вразливості смарт-контрактів. Оскільки саме смарт-контракти реалізують критично важливу бізнес-логіку системи та часто безпосередньо керують цифровими активами, будь-яка помилка в їхньому коді може мати серйозні наслідки. На відміну від звичайного серверного коду, який може бути оперативно виправлений після виявлення помилки, смарт-контракт після розгортання в мережі Ethereum часто стає або повністю незмінним, або змінюваним лише в межах спеціально передбачених механізмів оновлення. Це означає, що навіть незначна на перший погляд логічна помилка може зберігатися в системі тривалий час і бути використаною зловмисником без можливості швидкого реагування з боку розробника.

Серед найвідоміших і найбільш небезпечних вразливостей смарт-контрактів особливе місце займає reentrancy-атака. Її сутність полягає в тому, що під час виконання функції смарт-контракту зовнішній виклик до іншого контракту або адреси може призвести до повторного входу в ще не завершену функцію, якщо стан системи ще не був належним чином оновлений. Унаслідок цього зловмисник може багаторазово викликати логіку, яка повинна була виконатися лише один раз, і, наприклад, повторно виводити кошти до того, як баланс буде оновлено. Ця категорія вразливостей є особливо показовою, оскільки вона демонструє, що навіть формально правильний код може бути небезпечним, якщо розробник не враховує специфіку порядку виконання операцій у blockchain-середовищі.

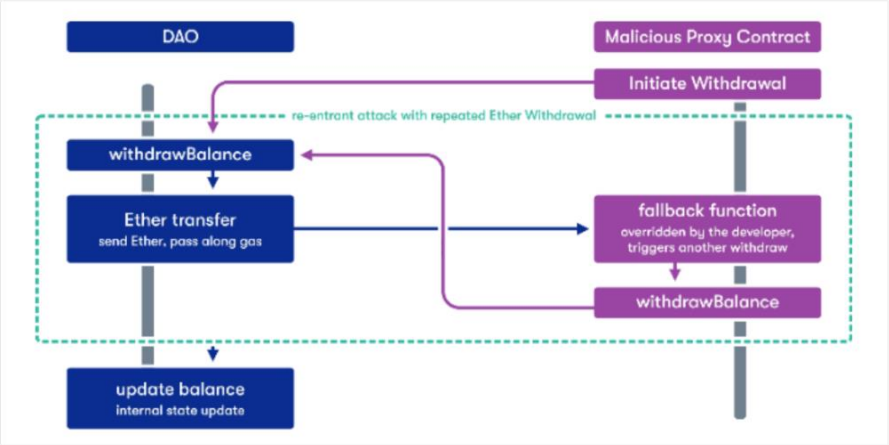

Рисунок 9. Механізм виконання атаки reentrancy

Не менш критичними є вразливості, пов’язані з помилками контролю доступу. У децентралізованих додатках функції смарт-контрактів часто мають різний рівень критичності: одні з них можуть бути доступними будь-якому користувачу, тоді як інші повинні виконуватися лише власником контракту, адміністратором або спеціально уповноваженими адресами. Якщо логіка перевірки таких повноважень реалізована некоректно або відсутня взагалі, зловмисник може отримати можливість змінювати критичні параметри системи, здійснювати несанкціоновані перекази, оновлювати конфігурацію або фактично перехоплювати контроль над додатком. У контексті Ethereum це особливо небезпечно, оскільки будь-який користувач мережі має технічну можливість напряму викликати публічні функції контракту, оминаючи клієнтський інтерфейс додатку. Таким чином, обмеження доступу не можуть покладатися лише на frontend і повинні бути реалізовані безпосередньо в логіці смарт-контракту.

Суттєву загрозу становлять також логічні вразливості, або flaws in business logic, які виникають не через технічні помилки мови програмування чи неправильну

реалізацію окремих операторів, а через хибну модель функціонування самої системи. Наприклад, смарт-контракт може дозволяти певну послідовність дій, яка формально не порушує його програмну логіку, але призводить до несправедливого або небезпечного результату. Такі помилки часто є складнішими для виявлення, ніж класичні баги, оскільки вони не завжди проявляються у вигляді аварійного завершення або некоректного виконання. Навпаки, система може працювати “як запрограмовано”, але при цьому залишатися вразливою до маніпуляцій із боку користувачів, які експлуатують непродумані бізнес-правила.

Окрему категорію становлять вразливості, пов’язані з неправильним використанням арифметичних операцій, особливо в контексті попередніх версій Solidity. Історично в Ethereum існували ризики переповнення та зменшення значень за межами допустимого діапазону, що могло призводити до неочікуваних змін балансу, порушення лічильників або обходу логіки перевірки. Хоча сучасні версії компілятора значною мірою зменшили ризик таких помилок завдяки вбудованим перевіркам, сама проблема арифметичної коректності залишається важливою, особливо якщо система використовує нестандартні математичні моделі, складні фінансові формули або оптимізовані обчислення. У децентралізованих фінансових додатках навіть незначна помилка в математичній логіці може мати прямий економічний ефект і бути використаною для отримання неправомірної вигоди.

Серйозною загрозою є також вразливості, пов’язані з небезпечною взаємодією між смарт-контрактами. У сучасній екосистемі Ethereum децентралізовані додатки рідко функціонують ізольовано. Вони часто інтегруються з токенами, зовнішніми протоколами, бібліотеками, проксі-механізмами, модулями управління або oracle-сервісами. Кожен такий зовнішній виклик розширює атакувальну поверхню системи. Якщо dApp беззастережно довіряє зовнішньому контракту, не перевіряє результат його виконання або не враховує можливість шкідливої поведінки, це може призвести до порушення цілісності логіки, втрати активів або компрометації критичних процесів. Таким чином, безпека dApp залежить не лише від якості власного коду, а й від безпечності всієї мережі взаємодій, у яку цей додаток інтегрований.

Рисунок 10. Механізм здійснення атаки на використання миттєвих позик

Окрему групу загроз формують економічні атаки, характерні саме для blockchain-середовища. На відміну від традиційних інформаційних систем, де основною метою атак часто є компрометація конфіденційності, цілісності або доступності даних, у децентралізованих додатках значна частина атак спрямована на отримання фінансової вигоди через використання властивостей протоколу або ринкових механізмів. Одним із найбільш показових прикладів є атаки, пов'язані з маніпуляцією ринковими параметрами, ліквідністю або часовими характеристиками транзакцій. Зловмисник може не ламати систему в класичному сенсі, а використовувати її власні правила проти неї самої. Саме тому для аналізу безпеки dApp недостатньо лише перевіряти відсутність технічних помилок - необхідно також оцінювати стійкість економічної моделі системи до зловживань. У межах цієї категорії особливої уваги заслуговують атаки, пов'язані з flash loans, або миттєвими позиками. Такі механізми дозволяють користувачу тимчасово отримати великий обсяг ліквідності в межах однієї транзакції без надання застави за умови, що позика буде повернена до завершення тієї ж транзакції. З одного боку, це є інноваційним фінансовим інструментом, а з іншого - відкриває можливості для проведення складних багатокрокових атак, у межах яких зловмисник може тимчасово маніпулювати ринковими параметрами, цінами, станом ліквідності або логікою протоколу. Якщо dApp не враховує подібні сценарії, він може виявитися вразливим до атак, які взагалі не мають аналогів у класичних web-системах.

Ще однією специфічною загрозою для децентралізованих додатків є front-running та інші форми маніпуляції порядком виконання транзакцій. У мережі Ethereum транзакції потрапляють до публічного пулу очікування до моменту їх включення в блок, що означає можливість спостереження за ними іншими учасниками мережі. Якщо зловмисник бачить транзакцію, яка може бути для нього вигідною, він може спробувати надіслати власну транзакцію з вищою комісією, щоб вона була виконана раніше. Унаслідок цього початкова логіка взаємодії може бути порушена. Наприклад, це може вплинути на децентралізовані біржі, аукціони, системи купівлі цифрових активів або протоколи, де важливим є порядок виконання дій. Така проблема є наслідком відкритості та конкурентності blockchain-середовища і вимагає окремих архітектурних механізмів захисту.

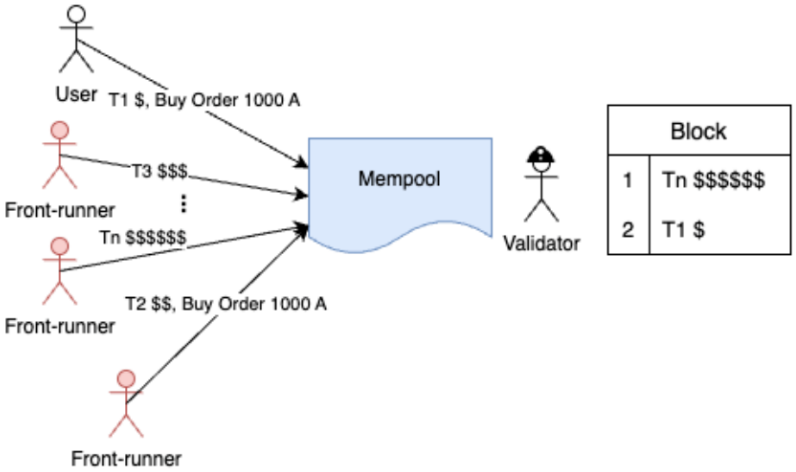

Рисунок 11. Схема здійснення front-running атаки через маніпуляцію чергою транзакцій у мемпулі

Суттєвим джерелом загроз є також використання зовнішніх джерел даних через oracle-механізми. Оскільки смарт-контракти не можуть самостійно отримувати інформацію із зовнішнього середовища, вони змушені покладатися на спеціальні сервіси або механізми доставки даних. Якщо oracle передає некоректну, затриману, маніпульовану або скомпрометовану інформацію, смарт-контракт може ухвалити помилкове рішення. Це особливо критично для фінансових додатків, де від зовнішніх цін, курсів, ставок або інших параметрів може залежати розподіл коштів, ліквідація позицій або розрахунок винагород. Таким чином, безпека dApp часто залежить не лише від внутрішньої логіки, а й від достовірності зовнішніх даних, на яких ця логіка базується.

Окремо слід розглянути загрози, пов'язані з користувацьким інтерфейсом децентралізованого додатку. Хоча смарт-контракти часто сприймаються як головний об'єкт аналізу безпеки, на практиці велика кількість атак відбувається саме через frontend-рівень. Оскільки dApp зазвичай має веб-інтерфейс, він успадковує класичні загрози web-безпеки, включаючи міжсайтовий скриптинг, ін'єкції в клієнтський код, компрометацію сторонніх бібліотек, підміну ресурсів, clickjacking або шкідливу зміну логіки відображення даних. У випадку dApp такі загрози можуть мати ще серйозніші наслідки, оскільки через скомпрометований frontend зловмисник може не просто викрасти дані сесії, а підштовхнути користувача до підпису шкідливої транзакції, яка після підтвердження буде виконана в блокчейні без можливості скасування.

Особливо небезпечною є загроза фішингу та обману користувача при підписі транзакцій. Оскільки у децентралізованих додатках критичні дії підтверджуються самим користувачем через гаманець, багато атак спрямовані не на злам смарт-контракту, а на маніпуляцію поведінкою користувача. Зловмисник може створити інтерфейс, який візуально імітує легітимний dApp, підмінити адресу контракту, показати оманливий опис операції або змусити користувача підписати не ту транзакцію, яку той очікував. У таких випадках з технічної точки зору мережа Ethereum не бачить жодної аномалії, адже дія була дійсно підписана власником ключа. Це означає, що безпека dApp значною мірою залежить не лише від захисту коду, а й від правильності та зрозумілості механізмів взаємодії з людиною.

Суттєві ризики виникають і на рівні зберігання та використання приватних ключів. У децентралізованих додатках користувачі самостійно контролюють свої ключі доступу, що відповідає ідеї децентралізації, але одночасно створює нові проблеми безпеки. Якщо ключ або seed-фраза компрометовані, користувач фактично втрачає контроль над своїми активами та правами в системі. На відміну від традиційних сервісів, де можна скинути пароль або звернутися до адміністратора, у blockchain-середовищі подібні механізми часто відсутні або суттєво обмежені. Це означає, що безпека dApp не може розглядатися ізольовано від безпеки криптографічної інфраструктури користувача, зокрема програмних і апаратних гаманців, середовища браузера, розширень та пристроїв.

Важливою категорією загроз є також проблеми конфігурації та розгортання децентралізованих додатків. Навіть якщо сам код смарт-контракту реалізовано коректно, помилки в процесі деплою можуть зробити систему вразливою. До таких

ризиків належать неправильна ініціалізація контракту, випадкове залишення тестових функцій у production-версії, помилкове призначення ролей адміністратора, використання неактуальних адрес зовнішніх сервісів, розгортання в неналежній мережі або помилки в конфігурації проксі-механізмів оновлення. У складних системах помилка не обов'язково має бути на рівні бізнес-логіки - інколи достатньо неправильно виконати початкове налаштування, щоб створити критичну вразливість.

Окремо варто підкреслити загрози, пов'язані з недостатнім журналюванням, моніторингом і виявленням інцидентів. У централізованих системах адміністратори зазвичай мають доступ до серверних логів, систем виявлення аномалій, журналів авторизації та інших засобів спостереження за роботою системи. У децентралізованих додатках ситуація складніша. Хоча транзакції та події смарт-контрактів є публічними, цього не завжди достатньо для своєчасного виявлення зловмисної активності, особливо якщо атака відбувається швидко, використовує легітимні функції контракту або поєднується з маніпуляцією зовнішніми сервісами. У результаті навіть публічність blockchain не гарантує автоматичного захисту, якщо система не супроводжується належними засобами аналізу та реагування.

Таким чином, загрози та вразливості децентралізованих додатків мають багаторівневий і міждисциплінарний характер. Вони охоплюють технічні помилки смарт-контрактів, хиби бізнес-логіки, проблеми контролю доступу, ризики взаємодії з зовнішніми контрактами та oracle-сервісами, економічні атаки, front-running, компрометацію клієнтського інтерфейсу, фішингові сценарії, ризики, пов'язані з приватними ключами, а також помилки конфігурації та недостатність моніторингу. Саме така багатовимірність загроз і визначає необхідність системного підходу до безпечної реалізації децентралізованих додатків. На відміну від традиційних web-систем, де безпека часто асоціюється насамперед із захистом сервера або бази даних, у dApp захищеною повинна бути вся екосистема взаємодії між користувачем, смарт-контрактом, blockchain-мережею та зовнішнім середовищем. Саме тому подальший аналіз вимог безпеки доцільно проводити з опорою на систематизовані підходи, зокрема стандарти та рекомендації OWASP, які можуть слугувати основою для побудови адаптованої моделі безпеки децентралізованих додатків.

РОЗДІЛ 2. АНАЛІЗ ВИМОГ OWASP ДО БЕЗПЕКИ WEB-ДОДАТКІВ

2.1. Призначення та структура стандарту OWASP TOP 10.

У сфері забезпечення безпеки програмних систем одним із найбільш відомих, авторитетних і практично значущих орієнтирів є стандарт OWASP Top 10, розроблений міжнародною некомерційною спільнотою OWASP. Цей документ посідає особливе місце серед рекомендацій із безпеки web-додатків, оскільки він не є вузькоспеціалізованим технічним стандартом лише для експертів із кібербезпеки, а виступає універсальним орієнтиром для розробників, архітекторів, тестувальників, аудиторів безпеки, менеджерів проєктів та організацій, що займаються створенням або експлуатацією програмних систем. Основне значення OWASP Top 10 полягає в тому, що він формує систематизоване уявлення про найбільш критичні ризики безпеки, характерні для сучасних web-додатків, і тим самим задає базовий рівень вимог до безпечної розробки програмного забезпечення.

Сам термін OWASP є скороченням від Open Worldwide Application Security Project - відкритої міжнародної ініціативи, метою якої є підвищення рівня безпеки програмних систем через поширення практичних знань, методичних матеріалів, відкритих інструментів, стандартів і рекомендацій. Особливістю діяльності OWASP є її відкритий, незалежний і некомерційний характер. Це означає, що запропоновані рекомендації не прив'язані до конкретних постачальників технологій або продуктів, а формуються на основі узагальнення реального досвіду, статистики вразливостей, аналітики інцидентів та експертних оцінок професійної спільноти. Завдяки цьому OWASP Top 10 розглядається не просто як інформаційний перелік поширених помилок, а як практичний орієнтир для побудови безпечного життєвого циклу розробки програмного забезпечення.

Основне призначення стандарту OWASP Top 10 полягає у формуванні єдиного зрозумілого переліку найбільш суттєвих ризиків безпеки web-додатків. Такий підхід має велике значення з практичної точки зору, оскільки у сфері інформаційної безпеки існує величезна кількість окремих типів вразливостей, помилок конфігурації, хибних архітектурних рішень і сценаріїв атак. Для організацій, які не мають великого досвіду у сфері AppSec, спроба одночасно охопити всі можливі загрози часто виявляється надто складною. Саме тому OWASP Top 10 виконує роль стартової, але водночас надзвичайно важливої моделі пріоритезації: він дозволяє зосередитися не на всіх теоретично можливих помилках, а на тих категоріях ризиків, які найчастіше призводять до реальних інцидентів або мають найвищий потенційний вплив на безпеку системи.

Важливо розуміти, що OWASP Top 10 не є вичерпним каталогом усіх вразливостей і не претендує на роль повної технічної специфікації для аудиту програмного забезпечення. Його призначення полягає не в тому, щоб замінити повноцінне моделювання загроз, архітектурний аналіз, тестування на проникнення чи формальну верифікацію, а в тому, щоб надати структуровану основу для розуміння найважливіших класів ризиків. У цьому сенсі OWASP Top 10 виконує не лише освітню, а й методологічну функцію: він допомагає організаціям формувати культуру безпечної розробки, впроваджувати базові захисні практики, визначати пріоритети для навчання команд і створювати початковий рівень зрілості процесів

AppSec.

Ще однією важливою функцією OWASP Top 10 є його роль як комунікаційного стандарту між різними учасниками процесу розробки та експлуатації програмних систем. У багатьох випадках проблеми безпеки виникають не лише через відсутність технічних засобів захисту, а й через те, що команди розробки, бізнес-аналітики, архітектори, DevOps-інженери та фахівці з безпеки по-різному розуміють характер ризиків і пріоритетність заходів захисту. OWASP Top 10 забезпечує спільну мову для опису типових категорій ризиків, що значно спрощує інтеграцію безпекових вимог у життєвий цикл розробки. Наприклад, коли в проєкті йдеться про ризик Broken Access Control або Security Misconfiguration, це вже не абстрактна проблема, а чітко окреслена категорія загроз із добре відомими прикладами, сценаріями експлуатації та рекомендаціями щодо запобігання.

Структурно OWASP Top 10 побудований як перелік десяти найбільш критичних категорій ризиків безпеки. При цьому важливо підкреслити, що кожен елемент списку є не окремою конкретною вразливістю, а узагальненою категорією, яка може охоплювати значну кількість технічно різних помилок, але об'єднаних спільною природою загрози або механізмом порушення безпеки. Саме завдяки цьому стандарт зберігає практичну цінність для широкого спектра систем і технологічних стеків, не прив'язуючись до окремих мов програмування, платформ або фреймворків. Такий підхід дозволяє використовувати OWASP Top 10 не лише як список для перевірки, а як концептуальну модель аналізу безпеки.

На сучасному етапі найбільш відомою та поширеною в професійному середовищі є редакція OWASP Top 10:2021, хоча OWASP уже також опублікував новішу редакцію 2025 року. Саме версія 2021 набула особливо широкого використання в навчальних матеріалах, аудитах безпеки, курсах із secure coding та внутрішніх корпоративних стандартах. У межах цієї редакції до переліку найбільш критичних категорій ризиків включено такі класи проблем, як Broken Access Control, Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration, Vulnerable and Outdated Components, Identification and Authentication Failures, Software and Data Integrity Failures, Security Logging and Monitoring Failures та Server-Side Request Forgery. Усі ці категорії разом формують узагальнену модель того, де саме найчастіше виникають серйозні проблеми безпеки в сучасних web-додатках.

Рисунок 12. OWASP Top 10 (2021)

Важливою особливістю структури OWASP Top 10 є те, що вона відображає еволюцію розуміння безпеки програмних систем. Якщо ранні підходи до AppSec часто концентрувалися переважно на окремих технічних помилках реалізації, наприклад ін'єкціях або проблемах автентифікації, то сучасна структура OWASP Top 10 значно ширша. Вона охоплює не лише дефекти кодування, а й архітектурні помилки, проблеми життєвого циклу розробки, ризики використання сторонніх компонентів, недоліки логування та моніторингу, а також порушення цілісності програмного забезпечення і даних. Це означає, що стандарт розглядає безпеку не як окремий технічний етап "перевірки коду", а як властивість усієї системи - від проєктування до експлуатації.

Особливо показовим у цьому контексті є включення до сучасної структури таких категорій, як Insecure Design та Software and Data Integrity Failures. Їх поява відображає перехід від вузького фокусу на експлуатаційні баги до ширшого бачення безпеки як результату якості архітектурних рішень, процесів розробки та ланцюга постачання програмного забезпечення. Наприклад, Insecure Design підкреслює, що навіть ідеально реалізований код не гарантує безпеки, якщо сама модель системи побудована хибно. Подібним чином Software and Data Integrity Failures акцентує увагу на ризиках, пов'язаних із використанням неперевірених компонентів, небезпечних механізмів оновлення, компрометацією залежностей або відсутністю контролю цілісності. Таким чином, структура OWASP Top 10 демонструє, що сучасна безпека додатків - це не лише боротьба з "класичними багами", а системна робота з усіма рівнями життєвого циклу програмного продукту.

Не менш важливою характеристикою стандарту є принцип його періодичного оновлення. OWASP Top 10 не є статичним документом; його редакції переглядаються та оновлюються з урахуванням нових тенденцій у сфері кібербезпеки, змін у практиці розробки, появи нових архітектурних моделей і накопичення статистичних даних щодо реальних вразливостей. Це означає, що OWASP Top 10 відображає не лише теоретичні уявлення про ризики, а й актуальну динаміку загроз у програмному середовищі. Зокрема, поява нових категорій або зміна позицій окремих ризиків у списку може свідчити про те, що певні типи проблем стають більш поширеними або більш критичними в умовах сучасної цифрової інфраструктури.

Рисунок 13. Порівняльний аналіз змін та еволюція категорій ризиків у стандарті OWASP Top 10 між редакціями 2021 та 2025 років

Окрему увагу слід приділити тому, як саме формується структура OWASP Top 10. Вона базується не лише на експертних оцінках, а й на аналізі великого масиву реальних даних про вразливості, інциденти та результати тестувань безпеки. Це дозволяє зменшити суб'єктивність при визначенні пріоритетних категорій ризику та підвищує практичну цінність документа. У цьому сенсі OWASP Top 10 можна розглядати як компроміс між аналітичною глибиною та прикладною доступністю: з одного боку, він достатньо простий для використання як навчальний або методичний інструмент, а з іншого - спирається на реальні емпіричні спостереження з галузі безпеки програмного забезпечення.

З точки зору практичного застосування OWASP Top 10 може використовуватися на кількох рівнях. На етапі проєктування він допомагає враховувати ключові класи ризиків ще до початку реалізації системи. На етапі розробки він слугує орієнтиром для впровадження захисних практик і перевірки правильності архітектурних рішень. На етапі тестування він може бути використаний як основа для побудови сценаріїв безпекового аналізу або penetration testing. На етапі експлуатації він допомагає організувати процес моніторингу, реагування на інциденти та періодичного перегляду безпекового стану системи. Таким чином, стандарт має цінність не лише як теоретичний документ, а як інструмент практичного управління ризиками в життєвому циклі програмного забезпечення.

Разом із тим, попри свою універсальність і високу прикладну цінність, OWASP Top 10 створювався передусім для класичних web-додатків і середовищ, де домінує клієнт-серверна модель взаємодії. Саме тому його пряме застосування до децентралізованих додатків не завжди є достатнім. Багато категорій ризиків, визначених у стандарті, безумовно залишаються актуальними й для dApp, зокрема ті, що стосуються контролю доступу, помилок конфігурації, криптографічних

механізмів, цілісності компонентів або безпечного проєктування. Однак у середовищі Ethereum ці ризики набувають нових форм і поєднуються зі специфічними загрозами, пов'язаними зі смарт-контрактами, незворотністю транзакцій, публічністю логіки, front-running, oracle-залежностями та економічними атаками. Саме тому в межах даної роботи OWASP Top 10 розглядається не як завершений і самодостатній перелік вимог для blockchain-систем, а як базова структурна модель, яка потребує подальшої адаптації до архітектури децентралізованих додатків.

2.2. Характеристика основних категорій вразливостей OWASP.

Стандарт OWASP Top 10 формує узагальнену модель найбільш критичних ризиків безпеки, характерних для сучасних web-додатків, і об'єднує їх у десять ключових категорій. Кожна з цих категорій не є окремою конкретною вразливістю, а представляє собою клас проблем, які можуть проявлятися в різних формах залежно від архітектури системи, технологічного стеку та особливостей реалізації. Такий підхід дозволяє розглядати безпеку не на рівні окремих технічних помилок, а як системне явище, що охоплює різні аспекти проєктування, розробки та експлуатації програмного забезпечення. У межах даного підрозділу доцільно розглянути основні категорії вразливостей, визначені у редакції OWASP Top 10:2021, з метою подальшого аналізу їх застосовності до децентралізованих додатків.

Однією з найбільш поширених і водночас критичних категорій є порушення контролю доступу. Ця категорія охоплює ситуації, коли користувач отримує можливість виконувати дії або отримувати доступ до ресурсів, які не відповідають його повноваженням. У традиційних web-додатках це може проявлятися у вигляді доступу до чужих облікових записів, перегляду або модифікації даних інших користувачів, виконання адміністративних функцій без відповідних прав або обходу механізмів авторизації. Проблема контролю доступу часто виникає не через складні технічні помилки, а через недосконалість або неповноту логіки перевірки прав доступу. Саме тому ця категорія стабільно займає провідні позиції у статистиці вразливостей, оскільки навіть незначні недоліки в реалізації можуть призводити до серйозних наслідків.

Не менш важливою є категорія криптографічних помилок, яка охоплює проблеми, пов'язані з неправильним використанням механізмів шифрування, хешування, зберігання секретів або передачі конфіденційної інформації. У сучасних системах криптографія є основою забезпечення конфіденційності, цілісності та автентичності даних, однак її неправильне застосування може повністю нівелювати захисні властивості. До типових помилок належать використання застарілих алгоритмів, неправильне управління ключами, відсутність шифрування там, де воно необхідне, або передача чутливої інформації у відкритому вигляді. Особливість цієї категорії полягає в тому, що навіть при формальній наявності криптографічних механізмів система може залишатися вразливою, якщо ці механізми застосовані некоректно.

Категорія ін'єкцій охоплює широкий спектр вразливостей, пов'язаних із неконтрольованою передачею даних користувача в інтерпретатор або виконавче середовище. Класичним прикладом є SQL-ін'єкція, однак у сучасних системах подібні атаки можуть проявлятися також у вигляді ін'єкцій у командний рядок,

LDAP, XML, шаблони або інші середовища обробки даних. Суть проблеми полягає в тому, що дані користувача сприймаються як частина коду або запиту, що дозволяє змінювати логіку виконання системи. Ін'єкції залишаються однією з найбільш небезпечних категорій вразливостей, оскільки вони часто дозволяють виконувати довільні дії в системі, отримувати доступ до конфіденційних даних або змінювати стан додатку.

Окрему категорію становить небезпечне проєктування, яка відображає перехід сучасної безпеки від аналізу окремих помилок реалізації до оцінки якості архітектурних рішень. Небезпечне проєктування означає, що система спочатку побудована таким чином, що навіть за відсутності явних програмних помилок вона не здатна забезпечити належний рівень захисту. Це може бути пов'язано з відсутністю моделювання загроз, неправильним визначенням довірених і недовірених зон, ігноруванням принципу мінімальних привілеїв або відсутністю механізмів контролю критичних операцій. Така категорія підкреслює, що безпека повинна закладатися на етапі проєктування, а не додаватися після завершення розробки.

Важливу роль відіграє також категорія помилок конфігурації безпеки. Вона охоплює випадки, коли система реалізована технічно правильно, але налаштована таким чином, що відкриває можливості для атак. До таких ситуацій належать використання стандартних облікових даних, неправильні налаштування серверів, відкриті порти, надмірні права доступу, відсутність обмежень на виконання операцій або використання небезпечних параметрів за замовчуванням. Проблеми конфігурації часто виникають у складних системах із великою кількістю компонентів, де кожен із них має власні параметри безпеки, і навіть одна помилка в налаштуванні може порушити захищеність усієї системи.

Категорія використання застарілих або вразливих компонентів відображає ризики, пов'язані із залежністю сучасних додатків від сторонніх бібліотек, фреймворків і сервісів. Більшість програмних систем сьогодні не створюються "з нуля", а базуються на великій кількості зовнішніх компонентів, які можуть містити відомі вразливості. Якщо такі компоненти не оновлюються або використовуються без належного контролю, вони стають точкою входу для атак. Ця проблема особливо актуальна в умовах швидкого розвитку технологій, коли нові вразливості виявляються регулярно, а процес оновлення компонентів не завжди встигає за цими змінами.

Окрему компоненту становлять помилки ідентифікації та автентифікації, які пов'язані з недоліками в механізмах підтвердження особи користувача. У традиційних web-додатках це може включати слабкі або передбачувані паролі, відсутність багатофакторної автентифікації, неправильне управління сесіями, відсутність обмежень на кількість спроб входу або неправильну обробку токенів доступу. Подібні вразливості можуть дозволити зловмиснику отримати контроль над обліковим записом користувача та виконувати дії від його імені. Особливість цієї категорії полягає в тому, що вона безпосередньо пов'язана з довірою до користувача та механізмами його ідентифікації.

Категорія порушення цілісності програмного забезпечення та даних охоплює ризики, пов'язані з неконтрольованими або неперевіреними змінами коду,

залежностей або даних. Це може включати компрометацію процесу оновлення програмного забезпечення, використання небезпечних або неперевірених джерел компонентів, відсутність механізмів перевірки цілісності або довіри до зовнішніх даних без належної валідації. У сучасному контексті ця категорія тісно пов'язана з проблемами безпеки ланцюга постачання програмного забезпечення, які набувають дедалі більшої актуальності.

Не менш важливою є категорія недостатнього журналювання та моніторингу. Вона відображає ситуації, коли система не забезпечує належного рівня спостереження за подіями, що відбуваються, або не має ефективних механізмів виявлення та реагування на інциденти. Навіть якщо атака відбулася, відсутність логів або їх недостатня деталізація може унеможливити аналіз події, виявлення причин або запобігання подальшим інцидентам. У результаті система може залишатися скомпрометованою протягом тривалого часу без виявлення проблеми.

Останньою категорією в межах OWASP Top 10:2021 є серверна підробка запитів, або SSRF. Ця вразливість дозволяє зловмиснику змусити сервер виконувати запити до внутрішніх або зовнішніх ресурсів, які зазвичай недоступні ззовні. У результаті це може призвести до отримання доступу до внутрішніх сервісів, витоку даних або виконання небезпечних операцій. Хоча SSRF не завжди є найбільш очевидною вразливістю, вона може мати серйозні наслідки в складних системах із розвиненою інфраструктурою.

Загалом усі розглянуті категорії формують цілісну модель загроз, яка охоплює різні аспекти функціонування web-додатків - від логіки доступу та обробки даних до конфігурації, використання компонентів і моніторингу. Важливо підкреслити, що ці категорії не є ізольованими одна від одної. У реальних системах вразливості часто комбінуються, і успішна атака може бути результатом поєднання кількох слабких місць, наприклад помилки конфігурації, відсутності перевірки доступу та використання вразливого компонента.

2.3. Аналіз можливості застосування вимог OWASP до децентралізованих додатків.

З урахуванням того, що стандарт OWASP Top 10 є одним із найбільш поширених орієнтирів у сфері безпеки web-додатків, природним є питання щодо можливості його застосування до децентралізованих додатків, побудованих на базі blockchain-платформ, зокрема Ethereum. Незважаючи на суттєві відмінності між традиційними клієнт-серверними системами та dApp, обидва типи програмних рішень мають спільні елементи, що дозволяє розглядати OWASP як базову модель для аналізу безпеки. Водночас специфіка децентралізованих систем вимагає критичного переосмислення окремих категорій ризиків та адаптації відповідних вимог до нових архітектурних умов.

Передусім варто зазначити, що децентралізований додаток не є повністю відокремленим від класичної web-архітектури. У більшості випадків dApp включає клієнтський web-інтерфейс, який реалізується за допомогою тих самих технологій, що й традиційні вебсистеми. Це означає, що значна частина вразливостей, описаних у OWASP Top 10, залишається актуальною на рівні frontend-компонента. Зокрема, такі категорії, як порушення контролю доступу, ін'єкції, помилки конфігурації або недостатній моніторинг, можуть безпосередньо проявлятися у клієнтській частині

децентралізованого додатку. Якщо інтерфейс реалізовано з порушенням базових принципів безпеки, це може призвести до компрометації взаємодії користувача із системою, навіть якщо смарт-контракти залишаються коректними.

Разом із тим, на відміну від класичних web-додатків, у dApp значна частина критично важливої логіки переноситься до смарт-контрактів, які виконуються в середовищі blockchain. Саме ця особливість суттєво змінює контекст застосування OWASP. Наприклад, категорія порушення контролю доступу в традиційній системі зазвичай пов'язана з перевірками на серверній стороні, тоді як у децентралізованому додатку такі перевірки повинні бути реалізовані безпосередньо в коді смарт-контракту. У цьому випадку недоліки контролю доступу набувають іншої форми: замість обходу серверної логіки мова йде про можливість виклику функцій контракту будь-яким учасником мережі без належної перевірки прав. Таким чином, сама категорія ризику залишається актуальною, але її технічна реалізація та механізми захисту змінюються.

Аналогічно можна розглянути категорію криптографічних помилок. У традиційних web-додатках вона стосується захисту даних під час передачі, зберігання паролів або використання шифрування. У децентралізованих додатках криптографія відіграє ще більш фундаментальну роль, оскільки вона лежить в основі всієї системи ідентифікації та авторизації. Замість логінів і паролів використовується модель цифрових підписів, а доступ до функціональності визначається володінням приватним ключем. У цьому контексті криптографічні помилки можуть проявлятися не лише в реалізації алгоритмів, а й у неправильному використанні ключів, небезпечній інтеграції з гаманцями або відсутності перевірки підписаних повідомлень. Таким чином, відповідна категорія OWASP залишається релевантною, але її зміст розширюється і поглиблюється.

Категорія ін'єкцій у децентралізованих додатках також зазнає трансформації. У класичному web-середовищі ін'єкції пов'язані з передачею небезпечних даних у запити до бази даних або інших інтерпретаторів. У dApp смарт-контракти не взаємодіють із SQL або подібними системами, однак ін'єкційні проблеми можуть виникати на рівні frontend, при формуванні запитів до RPC-вузлів або у взаємодії з off-chain компонентами. Крім того, певні аналогії можна провести з некоректною обробкою вхідних параметрів функцій смарт-контракту, що може призводити до логічних помилок або небажаної поведінки. Таким чином, хоча класична форма ін'єкцій змінюється, сама ідея небезпечної обробки вхідних даних залишається актуальною.

Особливої уваги заслуговує категорія небезпечного проєктування, яка є однією з найбільш універсальних і придатних до перенесення в контекст децентралізованих систем. У випадку dApp саме архітектурні рішення часто визначають рівень безпеки системи. Неправильно спроєктована логіка смарт-контракту, відсутність врахування сценаріїв зловживання, ігнорування можливості фронтранінгу або економічних атак можуть призвести до вразливостей, які неможливо усунути без повного перегляду архітектури. Таким чином, підхід OWASP, що акцентує увагу на важливості безпечного проєктування, є надзвичайно актуальним для dApp і навіть набуває більшого значення в умовах незмінності blockchain-коду.

Категорія помилок конфігурації безпеки також зберігає свою актуальність, хоча її прояви можуть бути іншими. У децентралізованих додатках конфігурація охоплює не лише налаштування серверів або вебсередовища, а й параметри розгортання смарт-контрактів, адреси зовнішніх сервісів, налаштування взаємодії з RPC-провайдерами, управління ролями в контрактах та використання проксі-механізмів оновлення. Неправильна конфігурація будь-якого з цих елементів може створити серйозні ризики для безпеки системи. Таким чином, хоча сама категорія залишається незмінною, її практичне наповнення значно розширюється.

Категорія використання застарілих або вразливих компонентів також є релевантною для dApp, оскільки сучасні децентралізовані додатки активно використовують сторонні бібліотеки, фреймворки, контракти-шаблони та інфраструктурні сервіси. Наприклад, використання застарілих версій бібліотек для роботи з blockchain або небезпечних шаблонів смарт-контрактів може створити додаткові вразливості. Крім того, залежність від зовнішніх контрактів або протоколів означає, що безпека dApp частково визначається безпекою цих компонентів.

Варіант помилок ідентифікації та автентифікації також потребує адаптації. У децентралізованих додатках відсутня класична модель облікових записів, однак це не означає відсутності ризиків. Навпаки, автентифікація переноситься на рівень криптографії, і будь-які помилки у взаємодії з гаманцями, обробці підписаних повідомлень або інтерпретації адрес можуть призвести до серйозних наслідків. Крім того, значну роль відіграє користувацький фактор, оскільки користувач самостійно підтверджує свої дії через підпис транзакцій.

Категорії, пов’язані з цілісністю програмного забезпечення та даних, а також журналюванням і моніторингом, також зберігають свою актуальність, хоча їх реалізація у dApp має інші особливості. Наприклад, у blockchain-середовищі частина інформації вже є публічною і доступною для аналізу, однак це не замінює необхідності побудови систем моніторингу та виявлення аномалій. Аналогічно, питання цілісності програмного забезпечення трансформується у питання довіри до смарт-контрактів, їхнього коду та процесу розгортання.

Водночас слід підкреслити, що не всі категорії OWASP можуть бути безпосередньо перенесені в середовище децентралізованих додатків без змін. Деякі з них, наприклад SSRF, мають обмежену застосовність у контексті смарт-контрактів, оскільки ті не взаємодіють із зовнішніми мережевими ресурсами у традиційному розумінні. Інші категорії, навпаки, потребують розширення, щоб врахувати нові типи загроз, характерні для blockchain-систем, зокрема front-running, reentrancy, економічні атаки або маніпуляцію зовнішніми даними через oracle-механізми.

Таким чином, аналіз показує, що OWASP Top 10 може бути використаний як базова модель для оцінки безпеки децентралізованих додатків, однак його застосування вимагає адаптації з урахуванням специфіки Ethereum та blockchain-архітектури. Частина категорій зберігає свою актуальність практично без змін, частина трансформується, а частина потребує доповнення новими типами загроз. Саме тому OWASP Top 10 слід розглядати не як готовий набір вимог для dApp, а як основу для формування розширеної моделі безпеки, яка враховує як класичні web-ризики, так і нові виклики децентралізованого середовища.

2.4. Обмеження класичних підходів OWASP у контексті blockchain-систем.

Попри високу практичну цінність стандарту OWASP Top 10 як універсального орієнтира у сфері безпеки web-додатків, його застосування до blockchain-систем і децентралізованих додатків має низку суттєвих обмежень. Ці обмеження пов'язані не з недосконалістю самого підходу OWASP, а з тим, що він був сформований передусім для аналізу ризиків у межах класичної клієнт-серверної архітектури, де центральне місце займають вебсервер, база даних, механізми сесійної автентифікації та централізоване управління бізнес-логікою. У середовищі blockchain значна частина цих припущень або взагалі не виконується, або набуває принципово іншої форми. Саме тому для коректного використання підходів OWASP у контексті Ethereum dApp необхідно не лише визначити їх релевантність, а й усвідомити межі їхньої застосовності.

Одним із головних обмежень класичних підходів OWASP є те, що вони орієнтовані на наявність централізованого контролю над системою. У традиційному web-додатку передбачається, що існує серверна частина, яка виконує логіку, зберігає дані, контролює доступ, перевіряє автентичність користувача та забезпечує можливість оперативного виправлення помилок. У blockchain-системах, зокрема на платформі Ethereum, логіка виконання переноситься до смарт-контрактів, які функціонують у децентралізованому середовищі без єдиного адміністративного центру. Це означає, що багато захисних механізмів, які в класичному web-середовищі реалізуються на серверному рівні, у dApp або повинні бути перенесені до логіки контракту, або взагалі не можуть бути реалізовані у звичній формі.

Ще одним суттєвим обмеженням є різниця в моделі довіри. Класичні підходи OWASP значною мірою виходять із того, що серверна інфраструктура контролюється довіреною стороною, а загрози здебільшого походять від зовнішніх користувачів, які намагаються обійти механізми захисту. У blockchain-системах модель довіри є набагато складнішою. Тут не можна безумовно вважати будь-який зовнішній компонент безпечним, а логіка системи повинна бути стійкою навіть у середовищі, де всі взаємодії є відкритими, публічними та доступними для аналізу будь-якому учаснику мережі. Замість припущення "сервер захищений, користувач потенційно небезпечний" у dApp фактично діє інший принцип: "будь-який зовнішній виклик, транзакція або взаємодія повинні вважатися потенційно ворожими, навіть якщо вони формально коректні". Такий підхід вимагає значно суворішого контролю над логікою виконання та глибшого врахування сценаріїв зловживання.

Обмеженням класичних підходів OWASP є також їхня орієнтація переважно на технічні помилки без достатнього акценту на економічну безпеку системи. У web-додатках порушення безпеки зазвичай розглядається через призму конфіденційності, цілісності та доступності інформації. У blockchain-системах, особливо у фінансово орієнтованих dApp, цього вже недостатньо. Тут навіть за повної відсутності класичних "багів" система може залишатися вразливою через хибну економічну модель, непродумані стимули, можливість маніпуляції цінами, ліквідністю або порядком виконання транзакцій. Класичний OWASP Top 10 практично не охоплює подібні сценарії, оскільки вони не характерні для більшості звичайних web-додатків.

Унаслідок цього застосування лише традиційного OWASP-підходу може створювати хибне відчуття безпеки, навіть якщо dApp залишається вразливим до економічних атак.

Окремим обмеженням є недостатня увага класичних підходів OWASP до незмінності логіки після розгортання. У традиційній вебархітектурі передбачається, що помилки можна оперативно виправляти шляхом оновлення серверного коду, зміни конфігурації або розгортання нової версії системи. У blockchain-середовищі ця модель працює зовсім інакше. Після розгортання смарт-контракту його код часто стає практично незмінним, а будь-яка помилка може зберігатися в системі протягом усього часу її існування. Це означає, що ціна помилки в dApp є значно вищою, ніж у звичайному web-додатку. Класичні підходи OWASP визнають важливість безпечного проєктування, але в контексті blockchain ця вимога стає ще більш жорсткою, оскільки багато помилок не можуть бути виправлені звичайними операційними засобами після введення системи в експлуатацію.

Ще одним важливим обмеженням є те, що OWASP Top 10 не враховує специфіку публічності blockchain-середовища. У класичних системах внутрішня логіка серверної частини, алгоритми обробки даних, умови перевірки доступу та інші критичні аспекти реалізації зазвичай приховані від зовнішнього користувача. У середовищі Ethereum код смарт-контракту, його байткод, адреса, виклики функцій і результати виконання доступні для аналізу. Це означає, що зловмисник має змогу детально досліджувати логіку системи ще до спроби її експлуатації. У такому середовищі недостатньо просто “не мати очевидних вразливостей” - система повинна бути стійкою до атак навіть за умови повної прозорості її логіки. Класичні підходи OWASP лише частково враховують цю особливість, оскільки вони формувалися для середовища, де внутрішня логіка додатку не завжди є публічною.

Не менш важливим є те, що OWASP Top 10 не охоплює повною мірою ризики, пов’язані з порядком виконання операцій. У традиційному web-додатку сервер зазвичай сам визначає послідовність обробки запитів і може забезпечувати відносно контрольовану модель конкурентного доступу. У мережі Ethereum транзакції надсилаються до публічного середовища та можуть бути включені до блоків у різному порядку залежно від ринкових механізмів, пріоритетів комісій або дій інших учасників. Саме тому з’являються такі специфічні загрози, як front-running, sandwich-атаки, маніпуляції черговістю виконання або експлуатація публічного mempool. Подібні проблеми практично не мають прямого аналога в класичному OWASP-підході, оскільки вони пов’язані не стільки з неправильним кодуванням, скільки з властивостями самої blockchain-інфраструктури.

Також недостатнє врахування міжконтрактної взаємодії та композиційності систем. У сучасній екосистемі Ethereum dApp часто взаємодіють не лише з власними смарт-контрактами, а й із токенами, протоколами ліквідності, механізмами управління, oracle-сервісами та зовнішніми контрактами третіх сторін. Така взаємодія створює складну мережу залежностей, у якій безпека одного компонента може прямо залежати від поведінки іншого. У класичних web-додатках також існує проблема сторонніх компонентів, але в blockchain-системах вона набуває принципово іншого масштабу, оскільки взаємодія відбувається не лише через API чи бібліотеки, а через виконання коду у спільному середовищі з безпосереднім впливом

на активи та стан системи. OWASP частково враховує ризики використання сторонніх компонентів, але не описує в повному обсязі ризики композиційної безпеки, характерні для DeFi та інших складних dApp.

Ще однією межею застосування класичних підходів OWASP є недостатня увага до специфіки користувацького підпису транзакцій. У web-додатках автентифікація та підтвердження дій часто відбуваються у формі введення пароля, токена або одноразового коду, а логіка контролю транзакцій повністю зосереджена на серверній стороні. У dApp користувач самостійно підписує транзакцію приватним ключем через гаманець, і саме цей підпис є юридично та технічно значущим підтвердженням дії. Це створює новий клас ризиків, пов'язаних не лише з компрометацією ключів, а й з маніпуляцією інтерфейсом, підміною змісту транзакції, фішингом та соціальною інженерією. У такому середовищі безпека значною мірою залежить від того, наскільки правильно організовано комунікацію між dApp, гаманцем і користувачем. Класичний OWASP не розглядає ці аспекти в повному обсязі, оскільки вони є специфічними саме для децентралізованих систем.

Класичні підходи OWASP не враховують повною мірою залежність blockchain-систем від зовнішніх джерел даних через oracle-механізми. У традиційному web-додатку сервер може напряму взаємодіяти із зовнішніми сервісами, перевіряти дані, повторювати запити або реалізовувати додаткову логіку захисту. Смарт-контракт такої можливості не має, тому змушений покладатися на спеціальні механізми доставки зовнішньої інформації. Якщо oracle працює некоректно або є об'єктом маніпуляції, навіть технічно правильний контракт може приймати небезпечні рішення. Класичні категорії OWASP не охоплюють подібний тип залежності як окремий системний ризик.

Також слід зазначити, що традиційні підходи OWASP не повністю відповідають вимогам до безпеки децентралізованих додатків із точки зору інцидент-менеджменту та відновлення після атак. У централізованих системах після інциденту можна заблокувати акаунт, відкликати токен, відкотити базу даних, обмежити доступ або тимчасово вимкнути частину функціоналу. У dApp подібні механізми або взагалі відсутні, або мають дуже обмежене застосування. Якщо транзакція вже підтверджена в блокчейні, її неможливо просто "скасувати" звичайними адміністративними засобами. Це означає, що класичні моделі реагування на інциденти потребують переосмислення, а вимоги до превентивного захисту стають значно жорсткішими.

Усе зазначене свідчить про те, що класичні підходи OWASP залишаються надзвичайно цінною методологічною основою для аналізу безпеки, однак у контексті blockchain-систем вони не можуть використовуватися як повністю завершений і самодостатній набір вимог. Їх слід розглядати як базовий каркас, який необхідно розширити та адаптувати з урахуванням специфіки смарт-контрактів, незмінності коду, економічної логіки, відкритості транзакційного середовища, oracle-залежностей, фронтранінгу, криптографічної моделі автентифікації та багатокомпонентної природи сучасних dApp.

РОЗДІЛ 3. ФОРМУВАННЯ АНАЛОГІЧНИХ ВИМОГ БЕЗПЕКИ ДЛЯ ДЕЦЕНТРАЛІЗОВАНИХ ДОДАТКІВ НА БАЗІ ETHEREUM

3.1. Адаптація вимог OWASP до архітектури Ethereum dApp.

Враховуючи результати попереднього аналізу, можна зробити висновок, що стандарт OWASP Top 10 є важливою методологічною основою для оцінки безпеки програмних систем, однак його пряме застосування до децентралізованих додатків є обмеженим. Це обумовлює необхідність адаптації відповідних вимог із урахуванням особливостей архітектури Ethereum dApp, яка включає клієнтський інтерфейс, механізми взаємодії з криптогаманцями, смарт-контракти, blockchain-мережу, RPC-вузли та допоміжні off-chain компоненти. Саме комплексний характер такої архітектури визначає необхідність багаторівневого підходу до безпеки, де кожен елемент системи повинен розглядатися як потенційна точка виникнення загроз.

Першим кроком адаптації вимог OWASP є переосмислення категорії контролю доступу в контексті децентралізованих систем. У класичних web-додатках контроль доступу реалізується на серверній стороні, де перевіряються ролі користувача, права доступу та допустимість виконання операцій. У середовищі Ethereum відсутній централізований сервер, а отже функції контролю доступу повинні бути реалізовані безпосередньо в смарт-контрактах. Це означає, що кожна функція, яка змінює стан системи або впливає на критичні параметри, повинна містити чітко визначені умови доступу, засновані на перевірці адреси викликача або інших параметрів. При цьому необхідно враховувати, що будь-який користувач мережі може напряму викликати функцію контракту, минаючи клієнтський інтерфейс, тому всі обмеження повинні бути реалізовані на рівні blockchain-логіки, а не лише у frontend.

Адаптація категорії криптографічних помилок також набуває особливого значення. У децентралізованих додатках криптографія є не лише допоміжним інструментом, а основою всієї моделі взаємодії. Вимоги до безпеки повинні включати коректну обробку цифрових підписів, перевірку автентичності транзакцій, безпечну інтеграцію з криптогаманцями, а також захист користувача від підпису шкідливих або неправильно сформованих повідомлень. Крім того, необхідно враховувати ризики, пов’язані з управлінням приватними ключами, навіть якщо вони формально знаходяться поза межами dApp. Це означає, що система повинна мінімізувати можливості для помилок користувача, зокрема через зрозуміле відображення змісту транзакцій і уникнення неоднозначних або складних для інтерпретації дій.

Категорія ін’єкцій трансформується у вимоги до безпечної обробки вхідних даних у всіх компонентах системи. Хоча смарт-контракти не взаємодіють із традиційними базами даних, вони приймають параметри викликів, які можуть впливати на їхню поведінку. У цьому контексті необхідно забезпечити валідацію вхідних даних, обмеження допустимих значень, перевірку коректності адрес, числових параметрів та інших аргументів функцій. Крім того, важливу роль відіграє безпечна взаємодія frontend із зовнішніми API та RPC-вузлами, де також можуть виникати ризики некоректної інтерпретації або підміни даних.

Категорія небезпечного проєктування у випадку dApp набуває центрального

значення. Архітектура системи повинна бути побудована таким чином, щоб враховувати специфічні для blockchain загрози ще на етапі проєктування. Це включає моделювання можливих сценаріїв атак, аналіз економічних стимулів, врахування конкурентності транзакцій, можливості front-running, маніпуляції станом системи та взаємодії між контрактами. Важливою вимогою є також принцип мінімальних привілеїв, який передбачає обмеження доступу до функцій лише необхідними правами, а також розділення критичної логіки на окремі модулі. У цьому контексті безпека перестає бути лише технічним аспектом реалізації та стає невід'ємною частиною архітектурного дизайну.

Категорія помилок конфігурації в децентралізованих додатках трансформується у вимоги до безпечного розгортання та налаштування системи. Це включає правильну ініціалізацію смарт-контрактів, коректне визначення адрес адміністративних ролей, налаштування параметрів взаємодії з зовнішніми сервісами, перевірку середовища розгортання та використання безпечних значень за замовчуванням. Особливу увагу слід приділяти процесу деплою, оскільки помилки на цьому етапі можуть бути незворотними. Крім того, необхідно враховувати ризики, пов'язані з використанням проксі-контрактів і механізмів оновлення, які можуть створювати додаткові точки входу для атак.

Адаптація категорії використання вразливих компонентів передбачає контроль над усіма залежностями dApp, включаючи бібліотеки для роботи з blockchain, шаблони смарт-контрактів, сторонні протоколи та інфраструктурні сервіси. Вимоги безпеки повинні включати перевірку актуальності використовуваних компонентів, аудит їхнього коду, аналіз відомих вразливостей і мінімізацію залежності від неперевірених джерел. Особливо це стосується смарт-контрактів, які взаємодіють із зовнішніми протоколами, оскільки їхня поведінка може безпосередньо впливати на безпеку всієї системи.

Категорія ідентифікації та автентифікації у децентралізованих додатках набуває принципово нового змісту. Оскільки автентифікація здійснюється через криптографічні підписи, вимоги безпеки повинні включати коректну обробку адрес користувачів, перевірку підписаних повідомлень, захист від повторного використання підписів і запобігання фішинговим атакам. Крім того, необхідно враховувати ризики, пов'язані з прив'язкою дій до конкретного користувача, оскільки в blockchain-середовищі відсутня традиційна модель сесій.

Порушення цілісності програмного забезпечення та даних трансформується у вимоги до перевірки коду смарт-контрактів, контролю процесу їх розгортання та забезпечення довіри до використовуваних компонентів. У децентралізованих додатках особливо важливо забезпечити, щоб код, який взаємодіє з користувачем, відповідав фактичному байткоду, розгорнутому в мережі. Крім того, необхідно враховувати ризики, пов'язані з маніпуляцією зовнішніми даними, зокрема через oracle-механізми.

Категорія журналювання та моніторингу також потребує адаптації. Хоча blockchain забезпечує публічність транзакцій, цього недостатньо для ефективного виявлення інцидентів. Необхідно впроваджувати додаткові механізми моніторингу подій смарт-контрактів, аналізу транзакційної активності, виявлення аномалій і реагування на підозрілі дії. Це дозволяє підвищити здатність системи до своєчасного

виявлення загроз, навіть у децентралізованому середовищі.

Водночас деякі категорії OWASP, такі як SSRF, мають обмежену застосовність у контексті смарт-контрактів, але можуть залишатися актуальними для off-chain компонентів dApp. Це ще раз підкреслює, що безпека децентралізованого додатку повинна розглядатися комплексно, з урахуванням усіх рівнів архітектури.

У результаті адаптації вимог OWASP до архітектури Ethereum dApp можна сформулювати ключовий принцип: безпека децентралізованого додатку повинна забезпечуватися на всіх рівнях системи - від користувацького інтерфейсу до смарт-контрактів і зовнішніх сервісів. При цьому класичні категорії ризиків залишаються актуальними, але їх реалізація та механізми захисту суттєво змінюються. Саме така адаптація дозволяє використати сильні сторони OWASP як базової моделі безпеки та водночас врахувати специфіку blockchain-середовища.

3.2. Вимоги безпеки до клієнтської частини децентралізованого додатку.

Клієнтська частина децентралізованого додатку є одним із найбільш критичних компонентів його загальної архітектури, оскільки саме через неї користувач взаємодіє з blockchain-середовищем, переглядає стан системи, ініціює дії та підтверджує транзакції. Попри те, що ключова бізнес-логіка dApp часто реалізується у смарт-контрактах, безпека клієнтської частини залишається надзвичайно важливою, адже саме на цьому рівні формується інтерпретація дій користувача та їхній зв’язок із реальними blockchain-операціями. У разі компрометації frontend-рівня навіть формально безпечний смарт-контракт може використовуватися в небезпечний спосіб, якщо користувачеві буде відображено неправдиву інформацію, підмінено адресу отримувача, приховано реальний зміст транзакції або нав’язано шкідливу послідовність дій.

Однією з базових вимог до безпеки клієнтської частини є забезпечення цілісності та автентичності коду, який завантажується до браузера користувача. Оскільки більшість dApp реалізуються як web-додатки, вони успадковують класичні ризики, пов’язані з підміною JavaScript-коду, компрометацією залежностей, шкідливими оновленнями сторонніх бібліотек, неправильним використанням зовнішніх CDN-ресурсів або маніпуляцією статичними файлами. У зв’язку з цим безпечна реалізація клієнтської частини повинна передбачати використання контрольованого ланцюга постачання фронтенд-компонентів, мінімізацію залежності від неперевірених сторонніх пакетів, контроль версій, аудит бібліотек, а також впровадження політик безпеки браузера, які ускладнюють виконання стороннього або ін’єкованого коду. У dApp ця вимога має особливу вагу, оскільки будь-яка модифікація frontend-коду може напряму впливати на поведінку користувача при взаємодії з його активами.

Наступною критичною вимогою є захист клієнтської частини від класичних web-вразливостей, зокрема міжсайтового скриптингу, небезпечної обробки даних, DOM-маніпуляцій, clickjacking та інших атак, що можуть змінити поведінку інтерфейсу або ввести користувача в оману. У контексті децентралізованих додатків міжсайтовий скриптинг є особливо небезпечним, оскільки наслідком такої атаки може бути не лише компрометація сесії чи інтерфейсу, а й ініціювання шкідливої взаємодії з криптогаманцем. Якщо зловмисний код змінює адреси контрактів, параметри викликів або відображуваний зміст дії, користувач може самостійно

підтвердити небезпечну операцію, не усвідомлюючи її реального значення. Саме тому безпечна реалізація frontend повинна передбачати сувору валідацію та екранування даних, контроль джерел контенту, обмеження динамічного виконання скриптів, а також загальне дотримання принципів secure frontend development.

Окрему увагу необхідно приділити вимозі достовірного відображення blockchain-стану. Клієнтський інтерфейс dApp не лише відображає звичайні UI-елементи, а фактично інтерпретує стан смарт-контрактів і blockchain-мережі для кінцевого користувача. Це означає, що будь-яке спотворення інформації на цьому рівні може призвести до хибного сприйняття системи. Наприклад, якщо інтерфейс відображає некоректний баланс, підмінює статус операції, неправильно інтерпретує дозволи на використання токенів або приховує критичні параметри транзакції, користувач може приймати рішення на основі недостовірної інформації. У зв'язку з цим однією з фундаментальних вимог безпеки є забезпечення точного, прозорого та перевірюваного відображення on-chain даних, без спрощень або прихованих трансформацій, які можуть спотворити їхній зміст.

Не менш важливою є вимога мінімізації довіри до клієнтської логіки. У традиційних web-додатках frontend часто виконує допоміжну роль і не вважається довіреним компонентом, тоді як усі критичні перевірки виконуються на сервері. У dApp ситуація є складнішою, оскільки частина логіки може бути розподілена між frontend і смарт-контрактом. У такому середовищі критично важливо, щоб клієнтський інтерфейс не виступав єдиним джерелом обмежень або контролю поведінки користувача. Жодна перевірка, яка має безпекове значення, не повинна покладатися виключно на frontend. Якщо певна дія повинна бути заборонена або обмежена, це має бути гарантовано смарт-контрактом, а не лише прихованою кнопкою або UI-логікою. Таким чином, вимога безпеки полягає не лише в коректності frontend-коду, а й у правильному розподілі довіри між клієнтським і on-chain рівнями.

Критично важливою вимогою до клієнтської частини є прозорість дій, які потребують підпису користувача. Оскільки будь-яка важлива операція в dApp зазвичай завершується підписом транзакції або повідомлення через гаманець, frontend повинен забезпечувати максимально зрозуміле відображення сутності цієї дії до моменту підпису. Користувач повинен мати можливість однозначно зрозуміти, що саме він робить: чи це звичайна авторизація, чи надання дозволу на використання токенів, чи безпосередній переказ активів, чи зміна параметрів у протоколі. Небезпечним є будь-який інтерфейс, який приховує технічну суть дії за спрощеними або оманливими формулюваннями. Саме тому до клієнтської частини dApp має висуватися вимога інтерфейсної чесності: усі критичні операції повинні бути відображені користувачу зрозуміло, повно та без навмисного заниження ризику.

Ще однією важливою вимогою є коректне управління адресами, ідентифікаторами мережі та контрактною інфраструктурою. У децентралізованих додатках помилка або підміна хоча б одного з цих елементів може призвести до виконання операцій не в тій мережі, не з тим контрактом або на користь не того адресата. Клієнтська частина повинна чітко контролювати, до якої мережі підключений користувач, з якими адресами контрактів відбувається взаємодія, чи відповідає середовище очікуваному production-оточенню, а також чи не відбулося

непередбачене перемикання між мережами. Крім того, інтерфейс повинен запобігати змішуванню тестових і production-даних, оскільки подібні помилки можуть створювати ілюзію коректної роботи при фактичній взаємодії з неправильним середовищем.

Окремий блок вимог стосується безпечної взаємодії frontend із зовнішніми RPC-вузлами, API та сервісами індексації. Більшість dApp не взаємодіють із блокчейном напряму на низькому рівні, а використовують RPC-провайдерів, сервіси читання подій, аналітичні API або зовнішні сховища. Це створює ризики, пов'язані з довірою до джерела даних. Якщо клієнтський інтерфейс беззастережно довіряє будь-якому зовнішньому API без перевірки або узгодження з on-chain станом, користувач може отримувати хибну інформацію. Саме тому однією з важливих вимог безпеки є мінімізація залежності від недовірених джерел даних, а також чітке розмежування того, які дані є критично важливими та повинні підтверджуватися on-chain, а які можуть використовуватися лише як допоміжні.

Важливою вимогою є також захист від UI-рівневих фішингових сценаріїв. Оскільки децентралізовані додатки часто використовують однакові шаблони взаємодії з користувачем, зловмисники можуть легко імітувати легітимний інтерфейс, копіювати структуру сторінок, підмінювати кнопки, назви токенів, адреси контрактів або відображувані дії. У зв'язку з цим клієнтська частина безпечного dApp повинна бути спроєктована так, щоб зменшувати можливість соціальної інженерії. Це включає використання стабільної ідентичності сервісу, зрозумілої навігації, передбачуваних шаблонів критичних дій, відображення перевірених адрес і попереджень у разі нетипових сценаріїв взаємодії. Хоча повністю усунути фішинг неможливо лише технічними засобами, безпечний інтерфейс повинен зменшувати ризик того, що користувач легко стане жертвою маніпуляції.

Ще однією важливою умовою є відсутність зберігання критичних секретів або конфіденційної логіки у клієнтському коді. Оскільки frontend dApp виконується в браузері користувача, будь-яка інформація, вбудована в його код або локальні механізми зберігання, потенційно доступна для аналізу або компрометації. Це означає, що клієнтська частина не повинна містити жорстко закодованих приватних ключів, секретних токенів доступу, внутрішніх адміністративних параметрів або будь-яких критичних механізмів, безпечність яких залежить від їхньої "прихованості". У контексті децентралізованих додатків цей принцип є особливо важливим, оскільки спроба "сховати" щось у frontend-коді суперечить самій природі відкритого клієнтського середовища.

Клієнтський інтерфейс dApp повинен коректно реагувати на ситуації, коли транзакція відхилена, мережа недоступна, RPC-вузол не відповідає, контракт повертає помилку або гаманець не підтримує потрібний сценарій взаємодії. Небезпечною є як надмірно технічна, так і надто спрощена обробка таких помилок. У першому випадку користувач не розуміє, що сталося, у другому - отримує хибне відчуття, що операція "пройшла успішно", хоча насправді вона не була підтверджена. Саме тому безпечний frontend повинен не лише коректно відображати помилки, а й робити це таким чином, щоб користувач міг прийняти безпечне подальше рішення.

Окремої уваги заслуговує вимога до безпечного оновлення клієнтської

частини. На відміну від смарт-контрактів, frontend dApp може змінюватися відносно легко, і саме це створює як перевагу, так і ризик. З одного боку, оновлення дозволяють оперативно виправляти помилки, покращувати інтерфейс і додавати захист. З іншого боку, будь-яке оновлення клієнтської частини потенційно може змінити логіку взаємодії користувача із системою. У зв'язку з цим важливою вимогою безпеки є контрольованість процесу оновлення, можливість відстеження змін, використання перевірених каналів доставки та уникнення ситуацій, коли користувач фактично взаємодіє з інтерфейсом, що був непомітно змінений у небезпечний спосіб.

Таким чином, вимоги безпеки до клієнтської частини децентралізованого додатку охоплюють не лише класичні аспекти web-безпеки, а й специфічні для Web3 ризики, пов'язані з відображенням blockchain-стану, інтеграцією з криптогаманцями, прозорістю транзакційних дій, правильністю роботи з мережами та контрактами, а також мінімізацією довіри до frontend-рівня. Безпечна клієнтська частина dApp повинна бути не лише технічно захищеною, а й спроєктованою таким чином, щоб не вводити користувача в оману, не створювати прихованих ризиків і не покладати на браузерний інтерфейс критичних безпекових гарантій. Саме тому frontend у децентралізованому додатку повинен розглядатися не як другорядний візуальний шар, а як повноцінний об'єкт захисту в загальній моделі безпеки системи.

3.3. Вимоги безпеки до взаємодії з криптогаманцями та підписом транзакцій.

У децентралізованих додатках на базі Ethereum взаємодія з криптогаманцем є центральним механізмом підтвердження дій користувача, що принципово відрізняє їх від традиційних web-систем. Якщо у класичних додатках автентифікація та авторизація здійснюються через логіни, паролі та сесійні механізми, то у dApp основою ідентифікації виступає криптографічна адреса, а всі критичні дії підтверджуються шляхом цифрового підпису транзакцій або повідомлень приватним ключем. Такий підхід забезпечує високий рівень криптографічної надійності, однак водночас створює новий клас ризиків, пов'язаних із взаємодією між користувачем, клієнтським інтерфейсом і blockchain-мережею.

Першою ключовою вимогою безпеки у цьому контексті є забезпечення прозорості та однозначності дій, які підписує користувач. Оскільки підпис транзакції є остаточним підтвердженням наміру, будь-яка неоднозначність або недостатня зрозумілість операції може призвести до виконання небезпечної дії без усвідомлення її наслідків. У зв'язку з цим клієнтський інтерфейс повинен відображати всі критично важливі параметри транзакції, включаючи адресу отримувача, суму, тип операції, взаємодію з конкретним смарт-контрактом і можливі побічні ефекти. Особливо важливо уникати ситуацій, коли складна або потенційно небезпечна дія маскується під просту операцію, наприклад "підключення гаманця" або "підтвердження входу". Таким чином, інтерфейс повинен забезпечувати інформовану згоду користувача, а не лише формальне підтвердження дії.

Другою важливою вимогою є захист від підпису шкідливих або підроблених транзакцій. У децентралізованих додатках значна частина атак спрямована не на злам смарт-контракту, а на маніпуляцію користувачем. Зловмисник може створити підроблений інтерфейс, підмінити адресу контракту або змінити параметри

транзакції таким чином, щоб користувач підписав дію, яка фактично призводить до втрати активів або передачі контролю. У зв'язку з цим важливою вимогою є забезпечення цілісності взаємодії між frontend і гаманцем, а також використання перевірених адрес контрактів, які не можуть бути змінені без явного попередження. Крім того, інтерфейс повинен попереджати користувача у випадках нетипових або потенційно небезпечних операцій.

Особливу увагу слід приділити вимозі безпечної обробки дозволів на використання токенів. У мережі Ethereum поширеною практикою є надання смарт-контрактам дозволу на керування токенами користувача через механізм approve. Хоча ця функціональність є необхідною для роботи багатьох dApp, вона створює значні ризики, якщо користувач надає надмірні або необмежені дозволи. У разі компрометації контракту або його логіки це може призвести до неконтрольованого списання коштів. Саме тому важливою вимогою є обмеження дозволів лише необхідним мінімумом, чітке відображення обсягу наданого доступу та можливість його відкликання. Користувач повинен розуміти, що він не лише виконує одноразову дію, а може надавати довготривалий доступ до своїх активів.

Ще однією важливою вимогою є забезпечення захисту від повторного використання підписаних повідомлень. У деяких сценаріях dApp використовують підпис повідомлення для автентифікації або підтвердження дії без створення транзакції. Якщо такі повідомлення не містять унікальних параметрів або часових обмежень, вони можуть бути повторно використані зловмисником для виконання небажаних дій. У зв'язку з цим необхідно впроваджувати механізми захисту від replay-атак, зокрема використання nonce, часових міток або одноразових повідомлень, що не можуть бути застосовані повторно.

Оскільки Ethereum має декілька середовищ, включаючи тестові та основну мережу, помилки у визначенні мережі можуть призводити до некоректної роботи dApp або втрати коштів. Клієнтський інтерфейс повинен перевіряти, чи відповідає поточна мережа очікуваному середовищу, та повідомляти користувача у разі невідповідності. Крім того, необхідно запобігати ситуаціям, коли користувач виконує транзакцію в неправильній мережі через відсутність явного попередження.

Суттєвою вимогою є також захист від фішингових атак, спрямованих на взаємодію з криптогаманцями. Зловмисники можуть створювати підроблені сайти або інтерфейси, які імітують легітимні dApp, і спонукати користувача підключити гаманець або підписати транзакцію. У таких випадках blockchain не може визначити, що дія є небезпечною, оскільки вона підписана справжнім користувачем. Це означає, що відповідальність за безпечну взаємодію частково лежить на самому dApp, який повинен мінімізувати можливість подібних атак через стабільну ідентичність сервісу, чітку навігацію та відсутність неоднозначних сценаріїв взаємодії.

Важливим аспектом є вимога мінімізації довіри до клієнтського середовища користувача. Оскільки взаємодія з гаманцем відбувається через браузер або мобільний пристрій, існує ризик компрометації середовища виконання, наприклад через шкідливі розширення, віруси або модифікований браузер. У зв'язку з цим dApp не повинен покладатися на клієнтське середовище як на повністю довірене. Усі критичні перевірки повинні виконуватися на рівні смарт-контракту, а клієнтська частина повинна розглядатися як потенційно вразлива.

Користувачі можуть використовувати програмні, апаратні або мобільні гаманці, кожен із яких має власні особливості. dApp повинен коректно працювати з усіма підтримуваними типами гаманців, не створюючи ситуацій, коли користувач змушений виконувати небезпечні або нестандартні дії. Наприклад, апаратні гаманці забезпечують вищий рівень безпеки, і додаток повинен підтримувати їх використання без втрати функціональності.

У середовищі Ethereum існує різниця між підписом транзакції, що змінює стан блокчейну, та підписом повідомлення, яке використовується для автентифікації або підтвердження дії поза блокчейном. Неправильне розуміння цієї різниці користувачем може призвести до небезпечних наслідків. Саме тому dApp повинен чітко розмежовувати ці типи операцій і пояснювати користувачу, яку саме дію він виконує.

Обмеження автоматизації взаємодії з гаманцем. Будь-які дії, що можуть призвести до зміни стану або втрати активів, повинні вимагати явного підтвердження користувача. Автоматичне підписання або приховані транзакції є неприйнятними з точки зору безпеки. Навіть якщо це знижує зручність використання, принцип явної згоди є критично важливим для забезпечення контролю над активами.

3.4. Вимоги безпеки до смарт-контрактів Ethereum.

Смарт-контракти Ethereum є центральним елементом децентралізованого додатку, оскільки саме вони реалізують критично важливу бізнес-логіку, визначають правила взаємодії між учасниками системи, зберігають стан і часто безпосередньо керують цифровими активами. На відміну від звичайної серверної логіки традиційного web-додатку, смарт-контракт функціонує в умовах публічного, незмінного та конкурентного середовища, де будь-яка помилка в коді може бути використана необмеженим колом осіб, а наслідки експлуатації вразливості часто мають незворотний характер. Саме тому до смарт-контрактів Ethereum повинні висуватися підвищені вимоги безпеки, які охоплюють не лише коректність реалізації коду, а й правильність логіки, стійкість до зловживань, надійність управління правами доступу, безпечність зовнішніх взаємодій і готовність до інцидентів.

Однією з базових вимог безпеки є принцип мінімізації складності логіки смарт-контракту. Чим складнішою є структура контракту, тим вищою стає ймовірність логічних помилок, прихованих вразливостей або неочевидних сценаріїв небезпечної взаємодії між його компонентами. У зв’язку з цим смарт-контракти повинні бути спроєктовані таким чином, щоб критичні функції були максимально простими, модульними та передбачуваними. Великі обсяги логіки доцільно розділяти на окремі компоненти, а допоміжні функції - виносити в окремі бібліотеки або модулі, за умови, що це не створює нових ризиків. Такий підхід знижує ймовірність появи складних помилок та полегшує аудит, тестування і формальну перевірку коду.

Ключовою вимогою є коректна реалізація контролю доступу. Оскільки будь-який користувач мережі Ethereum технічно може звернутися до публічної функції контракту, безпека системи не може грунтуватися на припущенні, що “небажаний виклик не буде здійснений”. Кожна функція, яка змінює стан системи, впливає на конфігурацію, управляє активами або виконує адміністративні дії, повинна мати

чітко визначені умови доступу. Це передбачає використання прозорої моделі ролей, перевірку повноважень на рівні смарт-контракту та мінімізацію привілеїв. У випадках, коли контракт має адміністративні функції, необхідно чітко визначати, хто і за яких умов може змінювати критичні параметри, призупиняти роботу системи, оновлювати компоненти або керувати фондами. Документація OpenZeppelin прямо підкреслює, що access control є критичним для безпеки smart contracts, оскільки від нього може залежати управління minting, governance, transfers та іншими критичними діями.

Окрему увагу слід приділити захисту від reentrancy-вразливостей, які історично є одними з найнебезпечніших у середовищі Ethereum. Безпечний смарт-контракт повинен бути спроєктований так, щоб зовнішні виклики не створювали можливості для повторного входу у функцію до завершення критичних змін стану. Для цього необхідно дотримуватися безпечного порядку виконання дій: спочатку перевіряти умови, потім змінювати внутрішній стан і лише після цього взаємодіяти із зовнішніми контрактами або адресами. Такий підхід відомий як Checks-Effects-Interactions і прямо рекомендується в офіційній документації Solidity як базовий механізм зниження ризику reentrancy. Додатково до цього можуть застосовуватися захисні механізми типу reentrancy guard або моделі pull payment, коли користувачі самостійно ініціюють отримання коштів, а не отримують їх автоматичним “push”-переказом.

Важливою вимогою є також безпечна обробка зовнішніх викликів. У середовищі Ethereum будь-який зовнішній контракт або адреса, з якою відбувається взаємодія, повинні розглядатися як потенційно недовірені. Це означає, що смарт-контракт не повинен беззастережно припускати коректність або безпечність поведінки зовнішнього компонента. Усі результати зовнішніх викликів повинні перевірятися, а сама логіка взаємодії повинна бути стійкою до помилок, винятків, відмов у виконанні або спроб маніпуляції з боку зовнішнього контрагента. Особливо це стосується інтеграцій із токенами, зовнішніми протоколами, bridge-механізмами, oracle-сервісами та іншими on-chain компонентами. Безпечний контракт не повинен “довіряти за замовчуванням” жодному зовнішньому виклику.

Не менш важливою є вимога до коректної валідації всіх вхідних параметрів. Смарт-контракт повинен розглядати будь-який зовнішній вхід як потенційно шкідливий або некоректний, навіть якщо він надходить від очікуваного користувача або сумісного додатку. Це означає, що всі числові параметри, адреси, масиви, часові значення, прапорці стану та інші вхідні дані повинні проходити логічну перевірку на допустимість, цілісність і безпечність. Недостатня валідація може призводити не лише до аварійного завершення виконання, а й до логічних помилок, обходу захисту або порушення інваріантів системи. Саме тому OWASP Smart Contract Top 10 окремо виділяє lack of input validation як один із критичних класів ризику для Web3-систем.

Ще однією принциповою вимогою є збереження інваріантів і логічної цілісності бізнес-моделі контракту. У безпечному смарт-контракті важливо не лише те, щоб код “не падав” і не містив технічних багів, а й те, щоб він завжди зберігав фундаментальні правила функціонування системи. Наприклад, баланс не повинен ставати від’ємним або неузгодженим, загальна кількість токенів не повинна

змінюватися непередбаченим чином, доступ до певних функцій не повинен змінюватися без відповідного процесу авторизації, а взаємозв'язки між станами системи повинні залишатися послідовними. У складних dApp саме порушення інваріантів часто стає джерелом критичних інцидентів, навіть якщо окремі функції реалізовані формально правильно. Саме тому безпечна розробка смарт-контрактів повинна включати не лише написання коду, а й явне визначення ключових логічних властивостей системи.

Окрему увагу слід приділити вимозі стійкості до маніпуляцій із цінами, зовнішніми параметрами та ринковими умовами. Для багатьох dApp, особливо у сфері DeFi, смарт-контракти приймають рішення на основі вартості активів, стану ліквідності, ринкових співвідношень або інших зовнішніх даних. У таких випадках безпечна реалізація повинна враховувати ризик price oracle manipulation, flash loan–facilitated attacks та інших сценаріїв, коли зловмисник не "ламає" код напряму, а тимчасово змінює економічне середовище так, щоб контракт сам виконав небезпечну для себе дію. Саме тому будь-яка логіка, що залежить від зовнішніх числових параметрів, повинна бути захищена від короткочасних аномалій, маніпуляцій ліквідністю, однотранзакційних атак або використання неперевірених джерел даних. OWASP Smart Contract Top 10 окремо виділяє oracle manipulation та flash-loan facilitated attacks як критичні сучасні ризики Web3.

Вимога до безпечної роботи з арифметичними операціями, лічильниками, масштабуванням величин та обчисленнями часток. Хоча сучасні версії Solidity вже включають автоматичні перевірки переповнення та зменшення за межі допустимого діапазону для стандартних арифметичних операцій, це не усуває повністю ризик логічно небезпечних обчислень. У фінансових протоколах особливо важливо враховувати точність обчислень, округлення, порядок множення і ділення, масштабування одиниць виміру, а також можливість маніпуляції параметрами, які впливають на розрахунок винагород, часток або штрафів. Безпечний контракт повинен бути спроєктований так, щоб математична модель залишалася стабільною навіть за граничних або аномальних значень.

Безпечне управління станом та порядком зміни стану. У blockchain-середовищі дуже важливо, щоб зміни стану відбувалися передбачувано, послідовно і без можливості некоректного проміжного стану, який може бути використаний у межах тієї ж транзакції або внаслідок конкурентної взаємодії. Якщо контракт у певний момент переходить у тимчасово небезпечний стан до завершення всіх перевірок або оновлень, це може створити вразливість. Саме тому порядок операцій, момент оновлення змінних, блокування повторних викликів і правильне завершення критичних секцій коду мають фундаментальне значення для безпеки.

Окремо слід виділити вимогу до безпечної реалізації механізмів оновлення та upgradeability. Багато сучасних Ethereum dApp використовують проксі-архітектури, які дозволяють змінювати логіку системи без втрати стану. Хоча це підвищує гнучкість та дозволяє виправляти помилки після розгортання, подібний підхід одночасно створює нові ризики. Якщо механізм оновлення реалізовано неправильно, зловмисник або навіть ненавмисна помилка адміністратора можуть призвести до втрати контролю над контрактом, повторної ініціалізації, підміни реалізації або повної компрометації логіки системи. Саме тому безпечний upgradeable-контракт

повинен мати жорстко визначену модель прав доступу, захист від повторної ініціалізації, контроль процесу оновлення та прозорий механізм управління змінами. OWASP Smart Contract Top 10 окремо виділяє proxy and upgradeability vulnerabilities як самостійну категорію ризику.

Не менш важливою є вимога до наявності механізмів безпечного аварійного реагування. У випадку виявлення підозрілої активності, експлуатації вразливості або аномальної поведінки контракт повинен, за потреби, мати змогу перейти в обмежений режим функціонування, який мінімізує подальші збитки. У практиці Ethereum це часто реалізується через pausable-механізми або обмеження критичних операцій у разі надзвичайної ситуації. Водночас така функціональність сама по собі повинна бути ретельно захищена, оскільки будь-який emergency-mechanism також створює додаткову точку довіри та потенційний вектор зловживання. OpenZeppelin прямо розглядає Pausable як типовий security mechanism для incident response у smart contracts.

Критично важливою вимогою є також забезпечення тестованості та перевірюваності смарт-контракту. Безпечний контракт не повинен існувати як "чорна скринька", яку лише сподіваються правильно написати. Його поведінка повинна бути перевірена за допомогою модульних тестів, інтеграційних сценаріїв, тестів на граничні умови, fuzzing-підходів, а для особливо критичних компонентів - за допомогою формальних або напівформальних методів верифікації. У контексті безпеки Ethereum цього недостатньо робити "після написання коду"; навпаки, вимога до перевірюваності повинна закладатися ще на етапі проєктування контракту. Саме тому SCSVS позиціонує безпечну розробку не лише як coding discipline, а як verification-driven process для smart contracts і dApps.

Ще однією фундаментальною вимогою є забезпечення сумісності безпеки з економічною моделлю протоколу. У традиційному програмному забезпеченні правильність коду часто оцінюється через його функціональність, продуктивність і стійкість до технічних помилок. У smart contracts цього недостатньо. Контракт може бути технічно коректним, але економічно небезпечним, якщо його стимули, штрафи, механізми винагород, ліквідації, емісії або розподілу прав створюють можливості для зловживань. Саме тому безпечна реалізація smart contract повинна перевірятися не лише як програмний код, а як формалізована економічна система, стійка до стратегічної поведінки користувачів, маніпуляцій і сценаріїв "раціональної атаки".

Важливою вимогою є також використання перевірених шаблонів та безпечних бібліотек замість написання критичних механізмів "з нуля" без достатньої потреби. Якщо для реалізації стандартних функцій, таких як управління ролями, токен-логіка, пауза, перевірка підписів або захист від reentrancy, уже існують широко перевірені та аудовані бібліотеки, їхнє використання є більш безпечним підходом, ніж самостійне створення аналогічної логіки без достатнього обгрунтування. Водночас це не означає, що сторонній код є безпечним "автоматично"; будь-яка бібліотека або шаблон також повинні проходити перевірку сумісності, актуальності та відповідності моделі загроз конкретного dApp.

3.5. Вимоги безпеки до розгортання, адміністрування та супроводу dApp.

Безпека децентралізованого додатку не може розглядатися виключно як властивість його програмного коду. Навіть за умови коректної реалізації клієнтської

частини та смарт-контрактів, реальний рівень захищеності системи значною мірою залежить від того, яким чином вона була розгорнута, хто і як управляє її критичними параметрами, як організовано процес оновлення, реагування на інциденти та довгострокового супроводу. У blockchain-середовищі ця проблема набуває особливої ваги, оскільки наслідки помилок, допущених на етапі розгортання або адміністрування, часто є незворотними або можуть бути усунені лише ціною складних технічних і організаційних дій. Саме тому вимоги безпеки до dApp повинні охоплювати не лише розробку, а й увесь операційний життєвий цикл системи.

Однією з базових вимог є забезпечення контрольованого та перевірюваного процесу розгортання смарт-контрактів. У безпечному середовищі деплой не повинен бути одноразовою “ручною” дією, що виконується без формалізованих процедур. Навпаки, процес розгортання повинен бути відтворюваним, задокументованим і таким, що дозволяє однозначно перевірити, який саме код був зібраний, якими параметрами він ініціалізований, у якій мережі розгорнутий і з якими зовнішніми адресами пов’язаний. У випадку dApp на базі Ethereum це особливо важливо, оскільки помилки на етапі деплою можуть включати неправильну адресу адміністратора, помилкову ініціалізацію ролей, некоректні параметри токенів, підключення до неправильних контрактів або розгортання в неналежному середовищі. Оскільки після розгортання багато аспектів контракту можуть стати незмінними, вимога до точності та контрольованості цього етапу є критичною.

Вимога до безпечного управління адміністративними повноваженнями. У багатьох децентралізованих додатках, попри загальну ідею децентралізації, зберігаються окремі адміністративні функції, пов’язані з оновленням контрактів, зміною параметрів протоколу, паузою системи, керуванням whitelist/blacklist-механізмами або виконанням екстрених дій. У зв’язку з цим критично важливо, щоб такі повноваження не концентрувалися в одній точці без належного контролю. Безпечна система повинна мінімізувати кількість надпривілейованих ролей, чітко розмежовувати обов’язки, обмежувати можливості одноосібного критичного втручання та забезпечувати прозорість адміністративних дій. У практичному сенсі це означає, що модель управління повинна бути спроєктована не лише з точки зору функціональності, а й з позиції стійкості до компрометації або помилки адміністратора.

Окрему увагу необхідно приділити вимозі безпечного зберігання та використання адміністративних ключів. У blockchain-середовищі володіння приватним ключем означає фактичний контроль над відповідною роллю або ресурсом, а отже компрометація ключа адміністратора може мати катастрофічні наслідки для всього dApp. У зв’язку з цим неприпустимим є використання слабко захищених або особистих гаманців для виконання критичних адміністративних функцій. Вимоги безпеки повинні передбачати використання окремих захищених ключів, бажано із застосуванням апаратних засобів зберігання, багатопідписних схем або інших механізмів, що знижують ризик одноточкової компрометації. Крім того, важливою є наявність чіткої політики управління ключами, яка визначає порядок їх створення, використання, ротації, резервування та відкликання.

Суттєвою вимогою є також використання багаторівневого або колективного управління для критичних операцій. У безпечній архітектурі dApp не повинно бути

ситуацій, коли одна особа або один ключ можуть безконтрольно змінити логіку системи, оновити реалізацію контракту, вивести активи або змінити критичні параметри протоколу. Саме тому важливим механізмом захисту є використання багатопідписних схем, governance-процедур, часових затримок на виконання критичних змін або інших механізмів стримування, які зменшують ризик як зловмисного втручання, так і випадкової помилки. У контексті Ethereum такі механізми мають не лише технічне, а й організаційне значення, оскільки вони підвищують довіру до системи та зменшують залежність від однієї особи чи команди.

Не менш важливою є вимога до безпечного управління оновленнями. Багато сучасних dApp використовують upgradeable-архітектури, які дозволяють змінювати логіку смарт-контракту після розгортання. Хоча це забезпечує гнучкість та дозволяє виправляти помилки, саме механізм оновлення стає одним із найбільш критичних об'єктів захисту. Безпечний процес оновлення повинен включати попередню перевірку нової версії, контроль сумісності зі старим станом, аудит змін, захищений механізм ініціалізації та чітко визначений порядок активації нової логіки. Будь-яке оновлення не повинно здійснюватися імпульсивно або без належного тестування, оскільки помилка в новій реалізації може мати ще серйозніші наслідки, ніж початкова вразливість.

Критично важливою є вимога до безпечного супроводу клієнтської частини dApp. На відміну від смарт-контрактів, frontend може змінюватися відносно легко, що створює окремий ризик: навіть якщо blockchain-логіка залишається незмінною, зміна інтерфейсу може радикально вплинути на поведінку користувача. У зв'язку з цим оновлення клієнтської частини повинні бути контрольованими, перевіреними та прозорими. Необхідно уникати ситуацій, коли користувач фактично працює з новою логікою інтерфейсу, не усвідомлюючи цього, особливо якщо зміни стосуються механізмів підтвердження дій, відображення транзакцій або взаємодії з криптогаманцем. У безпечному dApp frontend не може розглядатися як "другорядна оболонка", яка не потребує контролю змін.

Побудова системи моніторингу та спостереження за поведінкою dApp після введення його в експлуатацію. Попри публічність blockchain, сама по собі доступність транзакцій не гарантує ефективного виявлення інцидентів. Безпечний супровід dApp повинен включати моніторинг подій смарт-контрактів, відстеження аномальної активності, спостереження за нетиповими викликами функцій, аналіз різких змін у стані системи, виявлення підозрілих транзакцій або незвичних патернів взаємодії. Особливо це важливо для фінансових dApp, де атака може відбутися протягом кількох блоків і спричинити значні збитки ще до того, як команда встигне відреагувати. Саме тому безпечна експлуатація dApp повинна включати не лише логічну безпеку, а й операційну здатність до спостереження та раннього виявлення проблем.

Наявність підготовленого механізму реагування на інциденти. У традиційних системах реагування часто передбачає блокування акаунтів, вимкнення сервісів, зміну серверної логіки або відкликання доступів. У blockchain-середовищі ці можливості суттєво обмежені, тому механізм реагування повинен бути продуманий заздалегідь. Це може включати функції паузи, обмеження окремих дій, відключення

інтеграцій із зовнішніми сервісами, переведення системи в режим обмеженої функціональності або інші наперед передбачені процедури. Водночас такі механізми самі по собі повинні бути захищені, щоб не стати додатковим вектором атаки або інструментом зловживання з боку адміністратора.

Вимога до прозорості змін і керування життєвим циклом системи. Безпечний dApp повинен забезпечувати можливість відстеження того, які саме зміни були внесені до системи, коли вони були застосовані, ким були ініційовані та як вони впливають на безпековий стан додатку. Це стосується як смарт-контрактів, так і клієнтської частини, конфігурацій зовнішніх сервісів, змін ролей або параметрів системи. Прозорість у цьому контексті є не лише питанням зручності чи довіри користувачів, а важливим елементом безпекового контролю.

Проведення регулярного аудиту безпеки та повторної оцінки ризиків. Безпека dApp не є статичним станом, який можна "досягти один раз". Навіть якщо система була безпечною на момент запуску, з часом можуть з'являтися нові вразливості, змінюватися зовнішні залежності, ускладнюватися сценарії використання або зростати цінність активів, що обробляються додатком. Саме тому безпечний супровід повинен включати періодичний перегляд моделі загроз, аудит нових змін, повторне тестування критичних компонентів та оцінку того, чи не з'явилися нові ризики внаслідок розвитку системи або її середовища.

Важливо також враховувати вимогу до безпечного управління залежностями та інфраструктурними сервісами. Хоча dApp працює в blockchain-середовищі, він часто залежить від зовнішніх RPC-провайдерів, CDN-ресурсів, аналітичних сервісів, backend-модулів, систем індексації або off-chain компонентів. Якщо хоча б один із цих елементів буде скомпрометований або налаштований небезпечно, це може вплинути на роботу всієї системи. Саме тому безпечний супровід dApp повинен включати не лише контроль on-chain логіки, а й постійний аудит усіх допоміжних компонентів інфраструктури.

3.6. Узагальнений перелік вимог безпечної реалізації децентралізованих додатків на базі Ethereum.

На основі проведеного аналізу архітектури децентралізованих додатків, особливостей функціонування смарт-контрактів у мережі Ethereum, а також вивчення підходів OWASP до безпеки web-додатків можна сформувати узагальнений перелік вимог безпеки до децентралізованих додатків на базі Ethereum. Необхідність такого узагальнення обумовлена тим, що безпека dApp не може бути забезпечена окремими, ізольованими заходами на одному рівні системи. На відміну від класичних web-додатків, у децентралізованому середовищі захищеність визначається сукупністю архітектурних, криптографічних, програмних, організаційних та операційних рішень, які повинні узгоджено діяти в межах єдиної моделі безпеки. При цьому класичні підходи OWASP зберігають свою цінність як методологічна основа, однак для Ethereum dApp вони мають бути розширені з урахуванням вимог до smart contracts, Web3-взаємодії, key management, secure deployment і постійної верифікації системи.

Першою фундаментальною вимогою слід вважати принцип безпечного проєктування системи ще до початку її реалізації. Децентралізований додаток повинен створюватися з урахуванням моделі загроз, специфіки blockchain-

середовища, можливих сценаріїв зловживання, конкурентного характеру виконання транзакцій, ризиків front-running, помилок у бізнес-логіці та потенційних економічних атак. Безпека не повинна додаватися до готової системи постфактум, навпаки, вона має бути закладена в саму архітектуру додатку. Це означає, що ще на етапі проєктування повинні бути визначені довірені та недовірені компоненти, модель ролей, допустимі межі взаємодії між контрактами, логіка роботи з активами та механізми аварійного реагування. OWASP Top 10 2021 окремо виділяє Insecure Design як одну з базових категорій ризику, а SCSVS орієнтує розробку smart contracts на verification-driven та security-by-design підхід.

Другою вимогою є забезпечення багаторівневої безпеки всієї архітектури dApp. Захищеним повинен бути не лише смарт-контракт, а кожен логічний шар системи: клієнтський інтерфейс, взаємодія з криптогаманцем, blockchain-логіка, механізми доступу до мережі, off-chain компоненти, залежності та адміністративна інфраструктура. У цьому контексті децентралізований додаток має розглядатися як цілісна система, де компрометація одного елемента може поставити під загрозу всю модель безпеки. Ethereum-екосистема прямо описує dApp як поєднання frontend, smart contracts і мережевої інфраструктури, а не як ізольований контракт у блокчейні.

Третьою ключовою вимогою є реалізація контролю доступу безпосередньо на рівні смарт-контрактів. Будь-які критичні функції, що впливають на стан системи, активи, ролі або параметри протоколу, повинні мати чітко визначені умови доступу, які не можуть обходитися через frontend або зовнішні сервіси. Модель доступу повинна базуватися на принципі мінімальних привілеїв і чіткому розмежуванні прав між користувачами, адміністраторами, операторами та іншими ролями. Вимоги OWASP щодо Broken Access Control у контексті dApp трансформуються саме в цю вимогу, а OpenZeppelin окремо підкреслює критичну важливість access control для smart contracts.

Четвертою вимогою є безпечна реалізація смарт-контрактної логіки. Контракти повинні бути простими за структурою, модульними, перевірюваними та стійкими до найбільш критичних класів Web3-вразливостей, зокрема reentrancy, некоректної валідації вхідних параметрів, небезпечної взаємодії із зовнішніми контрактами, oracle manipulation, flash-loan facilitated attacks, proxy/upgradeability flaws та логічних помилок бізнес-моделі. Безпечна реалізація передбачає дотримання патерну Checks-Effects-Interactions, перевірку результатів зовнішніх викликів, захист від повторної ініціалізації, збереження інваріантів стану та використання перевірених шаблонів там, де це можливо. Такі вимоги прямо підтримуються документацією Solidity, OWASP Smart Contract Top 10 і SCSVS.

П’ятою вимогою є забезпечення безпечної взаємодії користувача з криптогаманцем. Будь-яка дія, яка потребує підпису, повинна бути зрозумілою, прозорою та однозначною для користувача. Інтерфейс не повинен приховувати зміст операції, маскувати критичні дії під “звичайне підтвердження” або спонукати до підпису транзакції без належного пояснення її наслідків. Особливо важливою є коректна обробка approve-подібних механізмів, коли користувач надає контракту право керувати своїми токенами. Система повинна мінімізувати надмірні дозволи, забезпечувати зрозуміле відображення їхнього змісту та, за можливості, підтримувати безпечніші підходи до авторизації дій. У контексті OWASP це

пов'язано з Cryptographic Failures, Identification and Authentication Failures та Insecure Design, але в Web3 ці категорії набувають нової форми.

Шостою вимогою є захист клієнтської частини dApp як повноцінного об'єкта атаки. Frontend не може сприйматися лише як візуальна оболонка, оскільки саме через нього користувач отримує уявлення про стан системи та формує рішення щодо підпису дій. Клієнтська частина повинна бути захищена від XSS, підміни ресурсів, компрометації залежностей, clickjacking, небезпечних оновлень та інших типових web-загроз. Крім того, вона повинна достовірно відображати on-chain дані, чітко показувати мережу, контрактні адреси та зміст критичних операцій. Це безпосередньо продовжує вимоги OWASP Top 10 до web-безпеки в контексті dApp.

Сьомою вимогою є мінімізація довіри до зовнішніх і допоміжних компонентів. Децентралізований додаток часто залежить від RPC-провайдерів, off-chain API, сервісів індексації, CDN-ресурсів, oracle-механізмів, сторонніх бібліотек і зовнішніх smart contracts. Кожен із цих компонентів може стати джерелом ризику. У зв'язку з цим система повинна чітко розмежовувати критичні on-chain дані та допоміжну інформацію, перевіряти зовнішні залежності, уникати беззастережної довіри до сторонніх джерел і регулярно переглядати безпеку компонентів, від яких залежить її функціонування. Такий підхід узгоджується з категоріями Vulnerable and Outdated Components та Software and Data Integrity Failures з OWASP Top 10.

Восьмою вимогою є безпечне розгортання і контрольований життєвий цикл оновлень. Процес деплою smart contracts повинен бути відтворюваним, перевірюваним і задокументованим. Необхідно забезпечити коректну ініціалізацію ролей, параметрів та зовнішніх адрес, виключити можливість випадкового розгортання у неналежному середовищі й гарантувати відповідність вихідного коду фактичному байткоду, що працює в мережі. Якщо додаток використовує upgradeable-архітектури, механізм оновлення повинен бути окремо захищений і підконтрольний. SCSVS прямо охоплює secure deployment, governance та upgrade risks як частину smart contract security lifecycle.

Дев'ятою вимогою є захищене управління адміністративними повноваженнями та ключами. Критичні функції не повинні залежати від одного приватного ключа або однієї особи без належних компенсуючих механізмів. Управління оновленнями, аварійними функціями, зміною конфігурацій та іншими привілейованими діями повинно виконуватися з використанням багатопідписних схем, колективного контролю, часових затримок або інших механізмів стримування. Ключі, що надають адміністративний доступ, повинні зберігатися з використанням підвищених заходів захисту, а їх використання має регулюватися чіткою політикою key management. Це відповідає як операційним вимогам SCSVS, так і загальним принципам secure governance.

Десятою вимогою є забезпечення постійного моніторингу, спостереження за подіями та готовності до реагування на інциденти. Публічність blockchain не замінює повноцінного моніторингу. Безпечний dApp повинен мати можливість виявляти аномальні транзакції, нетипові послідовності викликів, спроби експлуатації вразливостей, підозрілі зміни стану та інші ознаки атаки. Крім того, система повинна мати передбачені механізми обмеження шкоди в разі інциденту, наприклад паузу, контрольоване відключення окремих функцій або інші заздалегідь спроєктовані

сценарії. OpenZeppelin прямо підтримує використання Pausable як практичний security primitive, а OWASP Top 10 наголошує на важливості Security Logging and Monitoring Failures як окремої категорії ризику.

Одинадцятою вимогою є регулярна перевірка безпеки та повторна оцінка ризиків. Навіть якщо dApp був безпечним на момент запуску, з часом можуть змінюватися його залежності, ринкове середовище, поведінка користувачів, обсяг активів під управлінням і актуальні техніки атак. Саме тому система повинна проходити повторні аудити, тестування змін, fuzzing, аналіз граничних сценаріїв, перегляд моделі загроз та перевірку безпеки нових інтеграцій. У випадку Ethereum dApp безпека є не одноразовим етапом, а безперервним процесом супроводу і верифікації, що прямо узгоджується з підходом SCSVS до security verification lifecycle.

Отже, узагальнений перелік вимог безпеки до децентралізованих додатків на базі Ethereum охоплює безпечне проєктування, багаторівневий захист архітектури, жорсткий контроль доступу, безпечну реалізацію smart contracts, прозору взаємодію з криптогаманцями, захист клієнтської частини, контроль зовнішніх залежностей, безпечне розгортання, надійне управління ключами, постійний моніторинг та регулярну повторну оцінку ризиків. Сукупність цих вимог формує адаптовану модель безпеки, яка спирається на методологічну основу OWASP, але враховує специфіку Ethereum, Web3 та смарт-контрактного середовища. Саме така модель може слугувати практичною основою для аналізу, проєктування і безпечної реалізації сучасних децентралізованих додатків.

ВИСНОВКИ

У результаті виконання лабораторної роботи було досліджено особливості безпечної реалізації децентралізованих додатків на базі blockchain-платформи Ethereum та проаналізовано можливість застосування класичних підходів OWASP до середовища Web3. У межах роботи було встановлено, що децентралізований додаток є значно складнішою системою, ніж традиційний web-додаток, оскільки поєднує у собі клієнтський інтерфейс, механізми взаємодії з криптогаманцями, смарт-контракти, blockchain-мережу, зовнішні сервіси та off-chain інфраструктуру. Саме така багаторівнева архітектура формує розширену поверхню атаки та вимагає комплексного підходу до забезпечення безпеки. Іншими словами, захищеність dApp не може бути досягнута лише шляхом написання коректного смарт-контракту, а повинна забезпечуватися на всіх рівнях функціонування системи.

У роботі було встановлено, що стандарт OWASP Top 10 є ефективною та методологічно сильною основою для аналізу ризиків безпеки, оскільки він систематизує найбільш критичні категорії вразливостей сучасних web-додатків та формує загальну модель безпечного проєктування, реалізації та супроводу програмних систем. Було показано, що значна частина категорій OWASP, зокрема порушення контролю доступу, криптографічні помилки, небезпечне проєктування, помилки конфігурації, використання вразливих компонентів та недостатній моніторинг, зберігає актуальність і для децентралізованих додатків. Водночас у процесі дослідження було доведено, що пряме перенесення класичних вимог OWASP у blockchain-середовище є недостатнім, оскільки Ethereum dApp функціонують в умовах іншої архітектурної моделі, іншої моделі довіри та іншої логіки взаємодії між користувачем і системою. OWASP Top 10 залишається важливим орієнтиром для AppSec, але для Web3-середовища його необхідно адаптувати та розширювати з урахуванням smart contracts, dApp-архітектури та EVM-based систем.

Під час аналізу було визначено, що класичні підходи OWASP мають низку суттєвих обмежень у контексті blockchain-систем. Зокрема, вони не враховують повною мірою незмінність смарт-контрактного коду після розгортання, публічність blockchain-логіки, ризики front-running та маніпуляції порядком виконання транзакцій, специфіку oracle-залежностей, flash loan–орієнтовані атаки, композиційні ризики взаємодії між контрактами, а також особливу роль криптографічного підпису як основного механізму підтвердження дій користувача. У зв’язку з цим у роботі було обгрунтовано, що безпечна реалізація децентралізованого додатку повинна базуватися не лише на класичних web-підходах, а й на спеціалізованих вимогах, сформованих для smart contract security та blockchain application security. OWASP Smart Contract Security Verification Standard (SCSVS) і суміжні ініціативи прямо орієнтовані на такі системи та підтверджують доцільність окремого підходу до Web3-безпеки.

У практично-аналітичній частині роботи було сформовано адаптований перелік вимог безпеки до децентралізованих додатків на базі Ethereum. Було показано, що ці вимоги повинні охоплювати кілька критичних напрямів. По-перше, безпека повинна закладатися на етапі проєктування системи через моделювання загроз, аналіз сценаріїв зловживання та правильне архітектурне розмежування

довірених і недовірених компонентів. По-друге, критично важливим є захист клієнтської частини dApp, яка повинна бути стійкою до типових web-загроз, забезпечувати достовірне відображення blockchain-стану та не вводити користувача в оману щодо сутності виконуваних дій. По-третє, окремий акцент повинен робитися на безпечній взаємодії з криптогаманцями, прозорості підпису транзакцій, мінімізації надмірних дозволів та захисті від фішингових сценаріїв.

Найбільш технічно значущою частиною дослідження стало формування вимог безпеки до смарт-контрактів Ethereum. У роботі було обгрунтовано, що безпечний смарт-контракт повинен бути простим за структурою, модульним, перевірюваним, стійким до reentrancy, безпечним у зовнішніх викликах, коректним у роботі з арифметикою та інваріантами стану, а також захищеним від логічних і економічних атак. Особливу увагу було приділено контролю доступу, безпечному порядку зміни стану, коректній роботі з upgradeable-механізмами, зовнішніми oracle-даними та інтеграціями з іншими контрактами. Офіційна документація Solidity прямо рекомендує враховувати reentrancy, secure patterns і відомі compiler bugs, а OpenZeppelin надає перевірені шаблони для access control, pausable та reentrancy protection, що ще раз підтверджує необхідність системного підходу до безпеки смарт-контрактів.

Крім того, у роботі було встановлено, що безпечна реалізація dApp неможлива без належної організації процесу розгортання, адміністрування та супроводу системи. Було обгрунтовано, що навіть коректно реалізований смарт-контракт може стати небезпечним у разі помилок під час деплою, слабкого захисту адміністративних ключів, відсутності контролю над оновленнями, неналежного управління ролями або відсутності моніторингу аномальної активності. Саме тому вимоги безпеки повинні включати захищене управління ключами, використання багатопідписних схем для критичних операцій, контрольовані механізми оновлення, постійний моніторинг on-chain подій та готовність до реагування на інциденти. OWASP Top 10 наголошує на важливості Security Logging and Monitoring Failures, а SCSVS та OpenZeppelin практично підтримують ідею security-aware governance та incident response mechanisms для smart contracts і dApps.

Отже, за результатами виконаної роботи можна зробити загальний висновок, що безпечна реалізація децентралізованих додатків на базі Ethereum вимагає переходу від класичного вузького розуміння безпеки як “відсутності вразливостей у коді” до ширшої концепції багаторівневої захищеності всієї цифрової екосистеми додатку. У цьому контексті OWASP виступає важливою базовою методологією, однак для Web3-середовища вона повинна бути доповнена спеціалізованими вимогами, орієнтованими на smart contracts, blockchain-архітектуру, криптографічну модель взаємодії та операційну безпеку.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Ethereum // Wikipedia. URL: https://en.wikipedia.org/wiki/Ethereum
2. Blockchair. Ethereum Explorer. URL: https://blockchair.com/ethereum
3. Macr J. Centralized vs Decentralized vs Distributed: A Quick Overview. URL: https://medium.com/@juliomacr/centralized-vs-decentralized-vs-distributed-a-quick-overview-1f3bd17b8468
4. Etherscan. USDT Token Tracker. URL: https://etherscan.io/token/0xdac17f958d2ee523a2206206994597c13d831ec7#tokenTrade
5. Ethereum Foundation. Ethereum Virtual Machine (EVM). URL: https://ethereum.org/developers/docs/evm/
6. Wood G. Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper). 2014. URL: https://ethereum.github.io/yellowpaper/paper.pdf
7. Binance Academy. How Do Gas Fees Work on Ethereum? URL: https://www.binance.com/uk-UA/academy/articles/how-do-gas-fees-work-on-ethereum
8. Alchemy. Ethereum Gas Explained. URL: https://www.alchemy.com/docs/ethereum-gas
9. Ethereum Foundation. Gas and Fees: Technical Overview. URL: https://ethereum.org/developers/docs/gas/
10. Quantstamp. What Is a Re-Entrancy Attack? URL: https://quantstamp.com/blog/what-is-a-re-entrancy-attack
11. ResearchGate. Opportunity for Front-running Attacks Arising from Validators. URL: https://www.researchgate.net/figure/Opportunity-for-Front-running-Attacks-Arising-from-validators-Arbitrary-Transaction_fig1_382793666
12. Cyfrin. Price Oracle Manipulation Attacks with Examples. URL: https://www.cyfrin.io/blog/price-oracle-manipulation-attacks-with-examples
13. OWASP Foundation. OWASP Official Website. URL: https://owasp.org/
14. InfosecTrain. OWASP Top 10 Vulnerabilities 2021 Explained. URL: https://www.infosectrain.com/blog/owasp-top-10-vulnerabilities-2021-revealed
15. UVCyber. OWASP Top 10 2025 and Modern AppSec. URL: https://www.uvcyber.com/resources/blog/owasp-top-10-2025-and-what-it-means-for-modern-appsec-programs
16. Reddy Y. OWASP Top 10 2021: A Complete Guide for Beginners in Cybersecurity. URL: https://medium.com/@yeddulamohanareddy/owasp-top-10-2021-a-complete-guide-for-beginners-in-cybersecurity-05cd91c85321
17. Solidity Team. Security Considerations — Solidity Documentation. URL: https://docs.soliditylang.org/en/latest/security-considerations.html
18. OpenZeppelin. Security — OpenZeppelin Contracts Documentation. URL: https://docs.openzeppelin.com/contracts/4.x/api/security
19. Ethereum Foundation. Ethereum.org: The Complete Guide to Ethereum. URL: https://ethereum.org/
20. Buterin V. A Next-Generation Smart Contract and Decentralized Application Platform. URL: https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf

21. Vogelsteller F., Buterin V. ERC-20: Token Standard. URL: https://eips.ethereum.org/EIPS/eip-20
22. Zheng Z., Su J., Chen J., Lo D., Zhong Z., Ye M. DAppSCAN: Building Large-Scale Datasets for Smart Contract Weaknesses in DApp Projects. URL: https://arxiv.org/abs/2305.08456